

Structure Sparsification and Sparsity Estimation for Efficient Sparse Matrix Multiplication

Guanghao Liang

The Chinese University of Hong Kong, Shenzhen
China

222041034@link.cuhk.edu.cn

Yixiang Fang*

The Chinese University of Hong Kong, Shenzhen
China

fangyixiang@cuhk.edu.cn

ABSTRACT

Sparse matrices have been widely used to model the relationships between entities in many scenarios, such as friendship in social networks and user-item relationship in e-commerce networks. As a foundational operation in scientific computing, graph analysis and machine learning, sparse matrix multiplication (SMM), which aims to efficiently multiply two or more sparse matrices, has received tremendous attention. To enable efficient SMM, existing approaches often incorporate a symbolic phase that estimates the sparsity of the output matrix before numeric multiplication, which then allows for optimizing the memory allocation, workload balancing, and multiplication order. While many works have studied the sparsity estimation, they struggle to derive accurate estimation with high efficiency. Moreover, even with exact output sparsity, redundant computation remains, especially when multiplying three or more matrices. To address these issues, we first propose a novel concept CoreMat, a list of sparsified matrices with the minimum number of non-zeros (NNZ) whose multiplication yield the same output as the multiplication of original input matrices, by bridging the SMM problem with connectivity analysis in multi-partite graphs. We theoretically establish the equivalence property of CoreMat and its relationship to (1, 1)-core, a classical concept of cohesive subgraph, and propose an efficient algorithm to extract it. Based on CoreMat, we further propose Ranger, a sparsity estimator which achieves much higher accuracy in both theory and practice than prior estimators, with comparable efficiency. Extensive experiments on real-world datasets demonstrate the superior performance of Ranger compared to state-of-the-art (SOTA) methods.

RIC1.1

PVLDB Reference Format:

Guanghao Liang and Yixiang Fang*. Structure Sparsification and Sparsity Estimation for Efficient Sparse Matrix Multiplication. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/GuanghaoLiang/Ranger_2026.

1 INTRODUCTION

A sparse matrix is a matrix in which most of the elements are zero. Sparse matrices are widely utilized to model the relationships between entities, as real-world relationships prevalently exhibit

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Figure 1 illustrates Sparse Matrix Multiplication (SMM) and its CoreMat. Part (a) shows the multiplication of three sparse matrices A_1 , A_2 , and A_3 to produce the output matrix O . Part (b) shows the multiplication of the CoreMat matrices A_1^* , A_2^* , and A_3^* to produce the same output matrix O . The matrices are represented as 3x3 grids with non-zero elements (1) and zero elements (0).

Figure 1: An example of SMM and its CoreMat.

sparse patterns. For example, in Twitter [24], the following-follower relationships between users can be represented by a sparse square matrix M , where $M[i, j]$ denotes whether the i -th user follows the j -th user (i.e., if yes, $M[i, j] = 1$; otherwise, $M[i, j] = 0$). As each user only follows a certain number of users, which account for a very small fraction of users in Twitter, the matrix M is very sparse.

As a foundational operation in network analysis, data mining and machine learning models, *Sparse Matrix Multiplication (SMM)*, which aims to efficiently multiply two or more sparse matrices, has received tremendous attention [1, 18, 20, 23, 31, 36, 40, 47, 54]. Figure 1(a) shows an example of multiplying three sparse matrices $A_1 \times A_2 \times A_3$, where O is the output matrix and the zero elements are omitted. SMM has found applications in many areas. A well-known application is the training of Graph Neural Networks (GNN) [22, 27, 51], which accounts for 80% of the training time [50]. For example, in a k -layer SGC model [51], the training stage involves SMM that multiplies $(k - 1)$ sparse matrices, i.e., $A^k X \Theta$, where A , X , Θ are the graph adjacency matrix, feature matrix and weight matrix, respectively. Another classic application lies in the k -hop relationship extraction in heterogeneous networks like DBLP [17], which can be done by multiplying the sparse matrices that model the 1-hop relationships. For instance, we can extract author-topic relationship by $M_{AP} \times M_{PT}$, where M_{AP} and M_{PT} are the sparse matrices of author-paper and paper-topic relationship, respectively.

As shown in the literature [18, 31, 47], the efficiency of SMM highly relies on the matrix sparsity estimation, where the sparsity of a matrix is defined as the proportion of non-zero elements. In Figure 1, for example, the sparsity of the output matrix O is $\frac{2}{9}$ as there are 2 non-zero entries among its 9 entries. Based on the above sparsity definition, researchers have also introduced the concept of row-wise sparsity [14, 31, 46], which measures the proportion of non-zero elements in a certain row. When multiplying two matrices, the row-wise sparsity values of all the rows of output matrix are important indicators for achieving balanced workload in parallel

execution. For example in $A \times B = O$, the classic parallel SMM algorithm typically adopts the row-by-row approach [21] to facilitate parallelization, since the row-wise sparsity of each row of O largely determines the workload of computing it. However, the row-wise sparsity is unknown in advance, so the workload balance largely depends on the sparsity estimation. When multiplying three or more matrices, the estimated sparsity values of intermediate matrices are leveraged to optimize the parentheses [31, 46], i.e., the order to execute the SMM, as enumerating all the possible parentheses is computationally prohibitive. Different parentheses may result in significantly varied number of total multiplications and memory occupation of intermediate matrices, which is similar to the case of dense matrix multiplication. Therefore, accurate sparsity estimation plays a vital role in efficient SMM.

• **Prior works.** The general workflow of popular SMM methods, such as MKL [23], cuSPARSE [36] and Kokkos [40], often incorporate a symbolic phase to estimate the sparsity of output matrix, then use the estimated sparsity value to optimize the memory allocation and workload balance accordingly, and finally perform numeric multiplication. Several sparsity estimators have been designed, and they can be roughly classified into three categories: 1) The probability-based methods [8, 26, 31, 46] apply the inclusion-exclusion principle [7] at different scales such as matrix-scale [8], block-scale [26] and row-scale [31, 46]. 2) The sampling-based methods [4, 5, 16, 55] sample the rows and columns from the input matrices to derive sampled outer products, then apply naive [55] or hash based methods [4, 5] to estimate the sparsity. 3) The graph-based method [14] transforms the row-wise sparsity estimation problem to estimating the size of transitive closure, and then uses the sample minimum as an estimator to predict the size of transitive closure.

Nevertheless, the above estimators struggle to achieve good trade-off between efficiency and accuracy. The probability-based methods assume that the non-zero entries are uniformly and independently distributed within and across the input matrices, which simplifies the problem and makes these estimators work efficiently, whilst introduces much estimation error since the assumption is unlikely to hold in practice. In addition, the inclusion-exclusion principle also intrinsically makes strict statistical assumptions, consequently the theoretical result of these methods are often recognized as the worst case or upper bound analysis. The sampling-based methods either cannot converge to the true sparsity, or cause expensive computation overhead. Moreover, when estimating the sparsity of multiplying a chain of three or more sparse matrices, the above estimators either suffer from significant accuracy downgrade as the number of input matrices increases, or become inapplicable because they require the intermediate matrices to be calculated and then sampled, which is computationally costly. Currently, the graph-based method [14] achieves the best estimation accuracy for multiplying a chain of sparse matrices with theoretical guarantees, but it requires substantial computational overhead for problem transformation. We compare the existing sparsity estimators with respect to time complexity, granularity, boundedness and whether they maintain performance in estimating the chain product in Table 4. Hence, a key question is how to design a more accurate sparsity estimator while maintaining high efficiency?

Besides, even with exact output sparsity, the existing SMM methods are still inefficient, because they have to conduct many multiplications between non-zero elements and zero elements, leading to

Table 1: Comparison of existing sparsity estimators.

Estimator	Time complexity	Row	Bound	Chain
MetaAC [8]	$O(1)$	×	✓	✓
DMap [26]	$O(mnl/b^3)$	×	✓	✓
MNC [46]	$O(k + \text{nnz}(A, B))$	✓	✓	✓
RS-estimator [31]	$O(\text{nnz}(A, B))$	✓	✓	✓
LayeredGraph [14]	$O(d(\text{nnz}(A, B)))$	✓	✓	✓
Ranger (ours)	$O(\text{nnz}(A, B) + dl')$	✓	✓	✓

★ Note: the dimensions of A and B are $m \times n$ and $n \times l$ respectively; b is the block size; d is the number of samples used in Monte Carlo sampling; $k = \max(m, n, l)$ is the maximum dimension; l' is the number of non-zero columns in B^* , and $l' \leq l$; ✓ means that the bound is derived under extra assumptions or the sparsity estimator suffers from error propagation when estimating the output sparsity of multiplying more than two matrices.

huge redundant computation. For example, the intermediate result $(A_1 A_2)$ in Figure 1, $(A_1 A_2)[1, 3]$ is zero-ed out by the empty row $A_3[3, :]$ thereby it is not contributing to O . Therefore computing $(A_1 A_2)[1, 3]$ is redundant. In fact, for SMM, we only need to derive the non-zero elements in the output matrix, so we should try to avoid the multiplications between non-zero elements and zero elements. To achieve this, a natural idea is to sparsify the original input matrices by removing as many non-zero elements as possible, under the constraint that the output matrix must remain unchanged. In Figure 1, for example, we can sparsify A_1 , A_2 , and A_3 as A_1^* , A_2^* and A_3^* , respectively, as $A_1 \times A_2 \times A_3 = A_1^* \times A_2^* \times A_3^*$. Clearly, since A_1^* , A_2^* , and A_3^* have significantly less total NNZ than the original matrices, the SMM could be performed much faster in both senses of total memory accesses and floating point operations (FLOPs). While the sparsification results in significant improvement in efficiency, it is non-trivial to identify these removable elements, which are ubiquitous and might cause unnecessarily large intermediate memory occupation and computation overhead. Therefore, the second key question is how to efficiently identify the removable elements?

• **Our technical contributions.** To enable efficient SMM, in this paper we address the above two questions by exploiting the structural information of the input matrices. We first focus on the second key question, and propose a novel concept, called CoreMat, which corresponds to the matrices with minimum number of non-zero elements that yields the same product matrix. In Figure 1, $A_1^* \times A_2^* \times A_3^*$ are called the CoreMat of the SMM problem $A_1 \times A_2 \times A_3$. The matrices in CoreMat are not only sparser than their corresponding original matrices, but also enable us to circumvent the redundant memory accesses and multiplications between non-zero elements and zero elements. By mapping the SMM problem to multi-partite graph, we identify that CoreMat is essentially a (1, 1)-core, which belongs to a classic family of sub-graphs called D-core [33] in directed graphs. We theoretically establish the correctness of CoreMat and the graph theoretic relationship between CoreMat and the (1, 1)-core, and then propose an efficient algorithm to extract CoreMat with a time complexity linear to the total NNZ in the input matrices.

Based on CoreMat, we answer the first question by developing Ranger (Range estimator), an accurate yet efficient row-wise sparsity estimator with theoretical guarantee for multiplying two or more sparse matrices. We leverage the sample range, i.e., the difference between sample maximum and minimum, to derive a

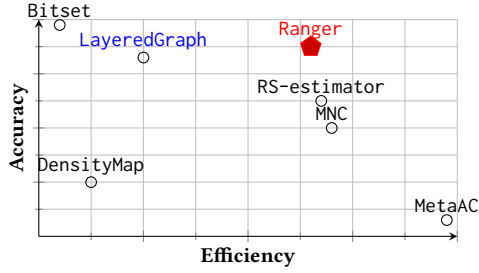


Figure 2: Comparison of accuracy and efficiency of sparsity estimators.

statistical estimator, which not only provides a theoretical convergence guarantee but also effectively captures the underlying structural properties of the sparse matrices. Figure 2 presents the accuracy and efficiency of Ranger in million-scale matrix sparsity estimations compared to prior sparsity estimators.

In summary, this paper makes the following contributions:

- For the SMM problem, we introduce the concept of CoreMat, the list of matrices with minimum number of non-zero elements that yield the same output matrix, and develop an efficient algorithm to derive CoreMat from input matrices.
- Based on CoreMat, we propose Ranger, a novel accurate yet efficient estimator for estimating the row-wise sparsity of the output matrix, with theoretical guarantee.
- Extensive experiments show that CoreMat is effective in sparsifying the input matrices and Ranger is much more accurate than SOTA competitors with comparable efficiency.

Outline. Section 2 introduces some preliminaries. Section 3 presents our proposed concept CoreMat, the theoretical analysis and its computation algorithm. Section 4 introduces our estimator Ranger, which builds upon CoreMat. Experimental results are reported in Section 5. We review related work in Section 6 and conclude the paper in Section 7.

2 PRELIMINARIES

Throughout this paper, we use L to denote the number of input sparse matrices in the SMM, where $L \geq 2$. We use $A_1, \dots, A_L$ and O to denote the input matrices and the output matrix, respectively. We use $A[i, j]$ to denote the element in the i -th and j -th column of the matrix A . We use the 1-base index in matrix elements for better alignment, i.e., $i, j > 0$. Table 2 summarizes the frequently used notations in this paper.

Table 2: Frequently used notations.

Notation	Description
$A[i, :]$	The i -th row in matrix A
$A[:, j]$	The j -th column in matrix A
$nnz(A)$	The number of non-zero elements in matrix A
$nnz(A[i, :])$	The number of non-zero elements in the i -th row
$\rho(A)$	The sparsity of matrix A , i.e., $\rho(A) = \frac{nnz(A)}{m \times n}$
$A_{i:j}$	The matrix product of $A_i \times A_{i+1} \cdots \times A_j$
$\eta(A, i)$	The sparsity of a row $A[i, :]$ in matrix A
$r(A)$	The vector of row-wise sparsity values of matrix A

In line with the design of existing sparsity estimators, we make the following no cancellation assumption [14, 26, 31, 46], which

is reasonable as multiplying floating-point matrices are unlikely to yield zeros due to the rounding error. For details on the no cancellation assumption, see our technical report’s Appendix [29].

Assumption 2.1 (No cancellation assumption). Given two input matrices A_1 and A_2 , the output matrix O satisfies: $O[i, j] \neq 0$ iff $\exists k \in \mathbb{Z}^+$ such that $(A_1[i, k] \neq 0) \wedge (A_2[k, j] \neq 0)$.

This assumption allows us to reduce the input floating-point matrices to boolean matrices as in Eq. (1).

$$A_{bool}[i, j] = \begin{cases} 1, & A[i, j] \neq 0 \\ 0, & A[i, j] = 0 \end{cases} \quad (1)$$

Next, we first introduce the sparse coordinate list storage format (COO), which is frequently used in SMM methods.

Definition 2.1 (COO format). Given a sparse matrix $A \in \mathbb{R}^{m \times n}$ with its NNZ denoted by $nnz(A)$, for any non-zero entry $A[i, j] = v$, we can represent it by a triplet $\langle i, j, v \rangle$. Consequently, the whole matrix A can be represented by a set $COO_A = \{\langle row_k, col_k, value_k \rangle\}_{k=1}^{nnz(A)}$ along with its dimension $m \times n$.

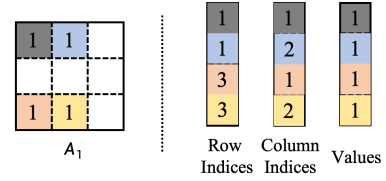


Figure 3: The COO format of A_1 in Figure 1.

In the following, we first present the problem of sparsity estimation, then introduce the general workflows of SMM, and finally introduce the graph representation of SMM problem.

2.1 Sparsity Estimation of SMM

Definition 2.2 (Sparsity and row-wise sparsity [31]). The sparsity of a matrix A of dimension $m \times n$ and the row-wise sparsity of its i -th row are respectively defined as

$$\rho(A) = \frac{nnz(A)}{m \times n}, \quad (2)$$

$$\eta(A, i) = \frac{nnz(A[i, :])}{n}. \quad (3)$$

For matrix A , its row-wise sparsity can be defined as a vector $r(A)$ by sequentially listing the row-wise sparsity of each row, i.e.,

$$r(A) = (\eta(A, 1), \eta(A, 2), \dots, \eta(A, m)) \quad (4)$$

Clearly, the sparsity of matrix A can be derived by $\rho(A) = \frac{1}{m} \sum_{i=1}^m \eta(A, i)$. Take the matrix O in Figure 1 as an example. Its sparsity $\rho(O) = \frac{2}{9}$, and its row-wise sparsity $r(O) = (\frac{1}{3}, 0, \frac{1}{3})$.

The sparsity estimation problem [26] is defined as follows.

PROBLEM 1 (ROW-WISE SPARSITY ESTIMATION). Given an SMM problem $A_1 \times A_2 \cdots \times A_L = O$, estimate the row-wise sparsity $r(O)$.

2.2 General Workflows of SMM

As shown in the literature, existing SMM methods can be classified into two categories based on the numbers of input matrices, i.e., $L = 2$ and $L \geq 3$. Next, we briefly introduce their workflows.

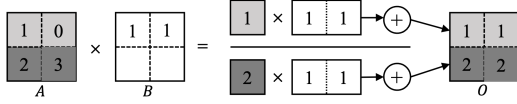


Figure 4: The workflow of Gustavson's algorithm.

• **SMM workflow for $L = 2$.** When $L = 2$, most SMM methods [23, 36, 40] follow a two-phase workflow. Specifically, they first estimate the row-wise sparsity in the symbolic phase, then perform parallel SMM multiplication in the numeric phase leveraging the Gustavson's algorithm [21], i.e., the row-by-row (RBR) approach.

We now briefly introduce the Gustavson's algorithm. Given an SMM problem $A \times B = O$, for every non-zero element in the i -th row of A , the algorithm uses the element's column index to select the corresponding row in matrix B . Each non-zero entry in this designated row of B is then multiplied by the non-zero value from A , and the resulting products are aggregated into the i -th row of the output matrix. Formally, the i -th row of O is computed as follows.

$$O[i, :] = \sum_{k \in \{k | A[i, k] \neq 0\}} A[i, k] \times B[k, :] \quad (5)$$

Example 1 illustrates the workflow of Gustavson's algorithm.

EXAMPLE 1. As shown in Figure 4, for the first row $A[1, :]$ in matrix A , since its entry $A[1, 1]$ is non-zero, the corresponding row $B[1, :]$ is multiplied via Eq. (5) to get the temporary result. For $A[2, :]$, both $A[2, 1]$ and $A[2, 2]$ are non-zero, the corresponding non-zero row $B[1, :]$ is multiplied with $A[2, 1]$. The output matrix O can be derived by aggregating the temporary results.

Clearly, Eq. (5) enables the row-wise parallel computation for $A \times B$. Besides, to balance the workload among rows in the parallel process, a widely adopted solution is to estimate the row-wise sparsity of output matrix O before multiplication.

• **SMM workflow for $L \geq 3$.** When $L \geq 3$, existing SMM methods follow another framework in Figure 5, since the order of the matrix multiplication has a significant effect on the efficiency.

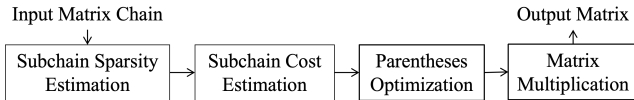


Figure 5: General SMM workflow for $L \geq 3$.

To be specific, they first estimate the sparsity of all $L(L-1)/2$ sub-chains. Then, for each sub-chain, they use the cost model in Eq. (6) to estimate the cost to perform SMM [26], where α , β , and γ are coefficients correlated to the hardware IO and GFlops, which can be calibrated by multi-variate regression. The parameters m , n , and l are the dimensions of matrices $A_1 \in \mathbb{R}^{m \times n}$ and $A_2 \in \mathbb{R}^{n \times l}$.

$$\text{cost}(A_1 A_2) \approx \alpha (m \cdot n \cdot \rho(A_1)) + \beta (m \cdot n \cdot \rho(A_1) \cdot l \cdot \rho(A_2)) + \gamma (m \cdot l \cdot \hat{\rho}(O)) \quad (6)$$

Next, they use dynamic programming to decide the optimal parentheses. Note that when $L \geq 3$, different parentheses, i.e., the orders to multiply these matrices, can cause drastically varied execution time, which is similar to the case in dense matrix multiplication. The number of legal parentheses equals to the Catalan number $C_{L-1} = (2L-2)! / (L!(L-1)!)$ [46, 48]. As enumerating all parentheses is computationally costly, people often leverage the estimated

sparsity of subchains to determine the optimal parentheses. Finally, they execute the SMM following the derived optimal parentheses.

2.3 Graph Representation of SMM

Based on Eq (1), we can map the multiplication of a chain of boolean matrices to a multi-partite graph via a bijection, where the multi-partite graph is a generalization of bipartite graph.

Definition 2.3 (Bipartite graph of a matrix). Given a sparse bool matrix $A \in \{0, 1\}^{m \times n}$, it can be mapped to a bipartite graph $G_A = (U, V, E)$ with a bijective mapping: for any $u_i \in U$, $v_j \in V$, u_i connects v_j iff $A[i, j] = 1$, where $|U| = m$, $|V| = n$, and $|E| = \text{nnz}(A)$.

Given an SMM problem, since A_l and A_{l+1} share the same multiply dimension, the mapped bipartite graphs can be concatenated to a $(L+1)$ -partite graph. Note that edges only exist between adjacent layers. We formally introduce it as follows.

Definition 2.4 (Multi-partite graph of SMM). Given an SMM problem $A_1 \times A_2 \cdots \times A_L$, its $(L+1)$ -partite graph, denoted by $G = (V, E)$, satisfying:

- $V = \bigcup_{l=1}^{L+1} V_l$, where V_l denotes the node set of the l -th partite (layer), and $\forall 1 \leq i, j \leq L+1$, $V_i \cap V_j = \emptyset$.
- $E = \bigcup_{l=1}^L E_l$, where E_l is the set of edges between V_l and V_{l+1} in the bipartite graph $G_{A_l} = (V_l, V_{l+1}, E_l)$.

Next, we present an important claim in Theorem 1 by Cohen [14], which establishes the relationship between the NNZ and reachable set size.

THEOREM 1 ([14]). Given an SMM problem $A_1 \times A_2 \cdots \times A_L = O$, the NNZ in the i -th row of O equals to the number of nodes in V_{L+1} that are reachable to the node v_i^1 in V_1 via some path(s) with L edges (i.e., the path must have one vertex from each V_l where $1 \leq l \leq L+1$).

We illustrate the above mapping mechanism via Example 2.

EXAMPLE 2. Consider the SMM problem $A_1 \times A_2 \times A_3 = O$ in Figure 1(a). Figure 6(a) shows its 4-partite graph representation $G_{A_1 \times A_2 \times A_3}$ and Figure 6(b) shows the bipartite graph representation $G_O = (V_1, V_4, E_O)$ of output matrix O . Clearly, the NNZ of the i -th row of O , i.e., $\text{nnz}(O[i, :])$, equals to the number of nodes in V_4 that connects to v_i^1 . For instance, assuming $i = 1$, we have $\text{nnz}(O[1, :]) = 1$ as v_1^1 connects to only one node v_4^1 in V_4 .

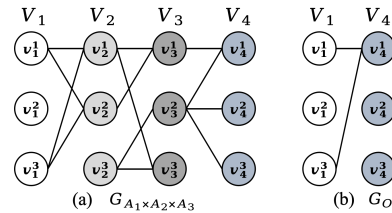


Figure 6: Multi-partite graph construction.

3 FORMULATION AND COMPUTATION OF COREMAT

As aforementioned, we observe that there exists a considerable number of non-zero elements that are zero-outed in numeric multiplication, especially when performing SMM for large sparse matrices. Removing these elements from the input matrices does not

change the output matrix, but significantly decreases the numbers of memory accesses and FLOPs, thereby improving the efficiency of SMM computation. However, identifying these elements is challenging since their positions in the input matrices are unknown in advance. A naive idea is to enumerate each non-zero element in one matrix and check whether it will be only multiplied with zero elements in other matrices. Obviously, this method takes a lot of time cost, which is almost the same as that of directly multiplying the sparse matrices, so it is impractical.

To efficiently identify the removable elements, we propose a novel concept, called CoreMat, which is a list of matrices with minimum number of non-zero elements that yields the same product matrix. In the following, we first formally introduce CoreMat, then establish its correctness via some theoretical analysis, and finally present a fast algorithm of computing CoreMat.

3.1 A New Concept: CoreMat

Given an SMM problem, we formally define its CoreMat as follows.

Definition 3.1 (CoreMat). Given an SMM problem $A_1 \times A_2 \cdots \times A_L = O$, its CoreMat is a list of matrices $A_1^*, A_2^*, \dots, A_L^*$ satisfying:

- **Subset property:** $\forall 1 \leq i \leq L$, $\text{COO}_{A_i^*} \subseteq \text{COO}_{A_i}$, and A_i^* shares the same matrix dimension as A_i .
- **Equivalence property:** $A_1^* \times A_2^* \cdots \times A_L^* = O$.
- **Minimality property:** For any other list of matrices, $A_1', A_2', \dots, A_L'$ that satisfies the above subset and equivalence properties, we have $\text{COO}_{A_i^*} \subseteq \text{COO}_{A_i'}, \forall 1 \leq i \leq L$.

Interestingly, CoreMat has three seminal features:

- (1) Its matrices are sparser than the original input matrices;
- (2) It is unique and minimal for a given SMM problem; and
- (3) When computing $A_1^* \times A_2^* \cdots \times A_L^*$, we only need to access and perform multiplications between non-zero elements.

Clearly, with these features, CoreMat allows us to perform both sparsity estimation and SMM computation more efficiently than those on the original input matrices.

EXAMPLE 3. Consider the SMM problem $A_1 \times A_2 \times A_3$ in Figure 1(a). Its CoreMat is a list of three matrices, A_1^*, A_2^*, A_3^* , in Figure 1(b). Clearly, these matrices are sparser than the original input matrices, but their multiplication yields the same output matrix O .

3.2 Theoretical Analysis of CoreMat

We first introduce a novel concept, called the D-multipartite graph.

Definition 3.2 (D-multi-partite graph). Given an SMM problem and its $(L+1)$ -partite graph G , we define its D-multi-partite graph as a $(L+3)$ -partite directed graph G^* , by adding two virtual nodes x, y , and some edges, such that:

- x has two directed edges linking to and from each node in the $(L+1)$ -th partite of G ;
- Each node in the 1-st partite of G has two directed edges linking to and from y ; and
- For each edge between two nodes in the l -th and $(l+1)$ -th partites with $l \in [1, L]$, we assign it a direction from the $(l+1)$ -th partite to the l -th partite.

Next, we show that the CoreMat of an SMM problem can be derived from its D-multi-partite graph using D-core [19, 33], which is a classic cohesive subgraph defined on directed graphs.

Definition 3.3 (D-core [19]). Given a directed graph $G = (V, E)$ and two integers $k, l \in \mathbb{Z}^+$, a D-core of G , denoted as the (k, l) -core, is a maximal subgraph $H = (V_H, E_H) \subseteq G$ such that the in-degree and out-degree of vertices in H are at least k and l , respectively.

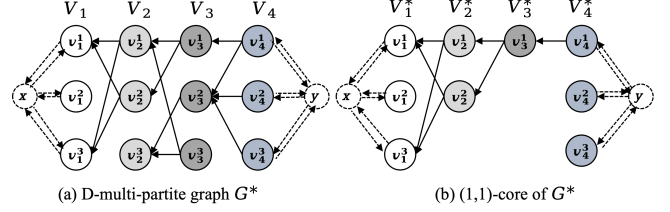


Figure 7: Illustrating D-multi-partite graph and its (1, 1)-core.

EXAMPLE 4. Consider the SMM problem in Figure 1. We depict its D-multi-partite graph in Figures 7(a), which is a 6-partite directed graph with virtual nodes x and y . Figure 7(b) presents the $(1, 1)$ -core of the D-multi-partite graph.

We formally establish the graph-theoretic relationship between CoreMat and the $(1, 1)$ -core of the D-multi-partite graph as follows.

THEOREM 2 (GRAPH-THEORETIC RELATIONSHIP). Given an SMM problem with L sparse matrices and its D-multi-partite graph G^* , the $(1, 1)$ -core of G^* , after removing s, t , shares the same edge set with the $(L+1)$ -partite graph of CoreMat.

PROOF. For the D-multi-partite graph G^* , denote its $(1, 1)$ -core by H , and the set of paths connecting the partites V_i and V_j by $\mathcal{P}_{i,j}$. From the perspective of outer product based SMM, we only need to prove that:

For any node $v \in G, v \in H \iff$ there exists a path $p \in \mathcal{P}_{1,L+1}$, such that $v \in p$.

We prove the case that $H \neq \emptyset$, since $H = \emptyset$ is trivial, i.e., the output matrix is zero.

$\Leftarrow$: We only need to prove $p \subseteq H$, then by $v \in p$ we can draw $v \in H$.

We first prove $p \cap H \neq \emptyset$ by raising a contradiction. Assume $p \cap H = \emptyset$. Since $p \in \mathcal{P}_{1,L+1}$, and edges in G only exists between adjacent layers, then each node $u \in p$ has in-degree ≥ 1 and out-degree ≥ 1 . Denote by $H' = H \cup p$. Noted that the original node degrees of H do not decrease by adding new subgraph (path) p to H . Thus, H' is the CoreMat and $H \subset H'$, which implies that H is not the maximal subgraph satisfying the degree condition of $(1, 1)$ -core in Def 3.3. By contradiction, we have $p \cap H \neq \emptyset$.

Next, we prove $p \subseteq H$ by contradiction. Assume $\exists v \in p$, such that $v \notin H$. Similarly, let $H' = p \cup H$, then H' is the CoreMat. Since $H \subseteq H'$ and $v \in H', v \notin H$, we have $H \subset H'$, which proves that H is not the maximal. By contradiction, we have $p \subseteq H$.

$\Rightarrow$: Without loss of generality, we assume $v \in V_l$, where $l \in [2, L]$. Since $v \in H$, we can traverse on both directions and sample one neighbor each time, i.e., the in-neighbor and out-neighbor of v in V_{l+1} and V_{l-1} , respectively. By continuing the traversal and transitivity property, we get a path $p \in \mathcal{P}_{1,L+1}$ and $v \in p$. In the case $l \in \{1, L+1\}$, we only need to traverse in one direction and apply similar analysis. $\square$

By proving Theorem 2, we essentially show that in the multipartite graph G of the SMM problem, every path connecting V_1

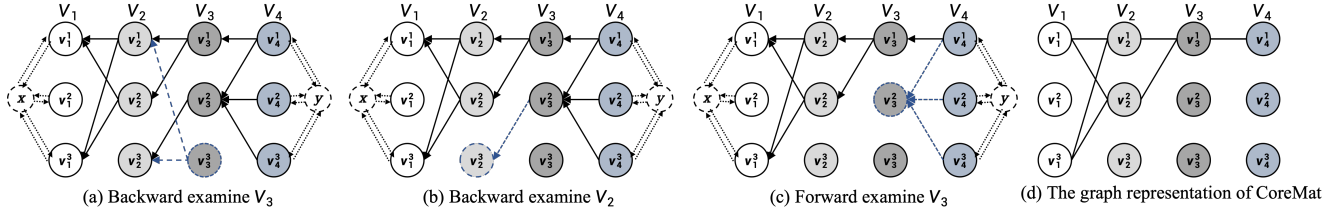


Figure 8: An example illustrating the CoreMat extraction algorithm CoreMat-Algo.

Algorithm 1: CoreMat-Algo.

Input: A sparse matrix chain $\{\text{COO}_{A_l}\}_{l=1}^L$

```

1 for  $l \leftarrow L, L-1, \dots, 2$  do
2    $\text{DV}(\text{row}_{A_l}), \text{DV}(\text{col}_{A_{l-1}}) \leftarrow$  distinct values in  $\text{row}_{A_l}, \text{col}_{A_{l-1}}$ ;
3    $S_l \leftarrow$  set intersection of  $\text{DV}(\text{row}_{A_l})$  and  $\text{DV}(\text{col}_{A_{l-1}})$ ;
4   for each index pair  $(i, j)$  in  $(\text{row}_{A_l}, \text{col}_{A_l})$  in parallel do
5     if  $i \notin S_l$  then Remove  $\langle i, j, v \rangle$  from  $\text{COO}_{A_l}$ ;
6   for each index pair  $(i, j)$  in  $(\text{row}_{A_{l-1}}, \text{col}_{A_{l-1}})$  in parallel do
7     if  $j \notin S_l$  then Remove  $\langle i, j, v \rangle$  from  $\text{COO}_{A_{l-1}}$ ;
8 for  $l \leftarrow 1, 2, \dots, L-1$  do
9    $\text{DV}(\text{col}_{A_l}), \text{DV}(\text{row}_{A_{l+1}}) \leftarrow$  distinct values in  $\text{col}_{A_l}, \text{row}_{A_{l+1}}$ ;
10   $S_l \leftarrow$  set intersection of  $\text{DV}(\text{col}_{A_l})$  and  $\text{DV}(\text{row}_{A_{l+1}})$ ;
11  for each index pair  $(i, j)$  in  $(\text{row}_{A_l}, \text{col}_{A_l})$  in parallel do
12    if  $j \notin S_l$  then Remove  $\langle i, j, v \rangle$  from  $\text{COO}_{A_l}$ ;
13  for each index pair  $(i, j)$  in  $(\text{row}_{A_{l-1}}, \text{col}_{A_{l-1}})$  in parallel do
14    if  $i \notin S_l$  then Remove  $\langle i, j, v \rangle$  from  $\text{COO}_{A_{l-1}}$ ;
15 return the updated sparse matrix chain  $\{\text{COO}_{A_l}\}_{l=1}^L$ ;

```

and V_{L+1} with L edges, i.e., the connectivity information between V_{L+1} and V_1 , is well preserved in H , whilst any node not lying on such a path is excluded. In other words, given an SMM problem, its multi-partite graph G , and the multi-partite graph H of its CoreMat, for any two nodes v_i^j and v_{L+1}^j in G , if they are connected via a path with L edges, then this path also exists between them in H . Therefore, the minimality and equivalence properties in Definition 3.1 are well guaranteed. Moreover, as the $(1, 1)$ -core is a sub-graph of the original graph, the subset property is naturally satisfied.

3.3 CoreMat Extraction Algorithm

By Theorem 2, a straightforward idea of computing CoreMat is to extract the $(1, 1)$ -core from the D-multi-graph of the SMM problem, but it is very costly since it is expensive to build the D-multi-graph for the SMM problem. To improve efficiency, we develop a fast algorithm which extracts CoreMat directly from the input sparse matrix chain based on set intersection.

Algorithm 1 shows the CoreMat extraction algorithm, called CoreMat-Algo, which takes a sparse matrix chain with each matrix stored in COO format [36] as input and outputs the CoreMat. It uses both backward (lines 1-7) and forward (lines 8-14) processes to extract CoreMat by leveraging the structural properties. Each iteration filters out those nodes that have zero in-degree or out-degree by taking intersection between column indices and row indices of adjacent matrices. Concisely, the backward process constructs a $(1, 0)$ -core, and the forward process constructs a $(0, 1)$ -core. The

output is equivalent to the intersection of $(1, 0)$ -core and $(0, 1)$ -core, therefore it is a $(1, 1)$ -core. The correctness of CoreMat-Algo is guaranteed by Theorem 3.

THEOREM 3. *Given an SMM problem, CoreMat-Algo correctly finds its CoreMat.*

PROOF. As CoreMat-Algo is essentially performing intersection between the forward and backward results, we only need to first prove that the backward and forward results are the $(1, 0)$ -core and $(0, 1)$ -core, respectively, and then prove that the intersection of $(1, 0)$ -core and $(0, 1)$ -core is a $(1, 1)$ -core.

Without loss of generality, here we only prove the backward result H_1 is the $(1, 0)$ -core, through which we can derive the forward result H_2 analogously, where H_2 is the $(0, 1)$ -core.

From the connectivity perspective, after the backward process, for any node $v \in H_1$, there exists a node $u \in V_{L+1}$ that v is reachable by u , i.e., H_1 is the set of all nodes that are reachable from V_{L+1} or in V_{L+1} . By similar analysis in Theorem 2, H_1 is maximal, therefore H_1 is the $(1, 0)$ -core.

Then, by using the maximal property, the node intersection $H_1 \cap H_2$ yields the maximal node set that have both in-degree and out-degree being at least 1, hence $H = H_1 \cap H_2$ is the $(1, 1)$ -core. $\square$

Example 5 illustrates the workflow of CoreMat-Algo.

EXAMPLE 5. *Consider the SMM problem in Figure 1. Figure 8 shows the workflow of extracting the CoreMat from its multi-partite graph. Notice that the intersection is equivalent to nodes with both in-degree > 0 and out-degree > 0 . In the backward process, we first examine V_3^* , as v_3^3 's in-degree is 0, and then remove edges connecting it (highlighted by dashed lines), as shown in Figure 8(a). Subsequently, we examine V_2^* , and then remove edges connecting v_2^3 , as depicted in Figure 8(b). In the forward process, we first examine V_3^* , and then remove edges connecting v_2^3 , as shown in Figure 8(c). Finally, we obtain the multi-partite graph of its CoreMat in Figure 8(d).*

We would like to remark that during the implementation, the temporary results and parallelize operations in Algorithm 1 are reusable by design. The $\text{DV}(\text{col}_{A_l})$ in line 9 equals to S_l in line 3. Also, the removal operations in lines 4, 6, 11 and 13 are **parallelizable** as they are independent, while removing the index pairs, for example in the backward phase, the $\text{DV}(\text{col}_{A_{l-1}})$ can be jointly derived during the scan. Hence, although Algorithm 1 uses some set operations, the latency of most of such operations can be mitigated and parallelized by intentional design.

LEMMA 4 (TIME COMPLEXITY OF ALGORITHM 1). *The time complexity of CoreMat extraction is linear to the total number of non-zeros of the input matrices, i.e., $O(\sum_{l=1}^L \text{nnz}(A_l))$.*

PROOF. The algorithm consists of two main loops:

- (1) The first loop processes each matrix pair (A_l, A_{l-1}) . For each iteration, computing distinct values and their intersection takes $O(\text{nnz}(A_l) + \text{nnz}(A_{l-1}))$ time, and filtering elements also requires linear time in the number of non-zeros.
- (2) The second loop similarly processes each matrix pair (A_l, A_{l+1}) in $O(\text{nnz}(A_l) + \text{nnz}(A_{l+1}))$ time per iteration.

Summing over all iterations, each $\text{nnz}(A_l)$ appears at most twice in the total cost. Hence, the overall time cost is $O\left(\sum_{l=1}^L \text{nnz}(A_l)\right)$. $\square$

R3C3.1

Remark: As shown in the literature [9], SMM and acyclic join query belong to the class of mathematical problems, called *Generalized Sum-Product Problem over Semirings* [9, 44]. A well-known algorithm for solving this problem is the Yannakakis algorithm [53]. To improve the efficiency, it uses two-pass reduction to remove the “dangling tuples” in the join process. Actually, the removable entries that are in the original input matrices but not in CoreMat correspond to the “dangling tuples” under the Semiring framework.

Although the extraction of CoreMat shares a reduction philosophy with the Yannakakis algorithm [53], CoreMat introduces three distinctive contributions: (1) CoreMat provides a graph-theoretic relationship with D-core [33]: for an input matrix chain, we theoretically prove that its CoreMat shares the same set of edges with the $(1, 1)$ -core of its multipartite graph. (2) Given the above theoretical grounding, established serial, parallel, and distributed D-core algorithms, including peeling-based [25] and H-index based methods [30], can be adapted for computing CoreMat. In other words, we do not need to solely rely on “two-pass reduction” idea of Yannakakis algorithm. (3) Yannakakis-style reduction is typically implemented via relational semi-joins, using sorting or hashing over tuples. In contrast, CoreMat operates directly on COO-format sparse matrices and replaces such relational operators with lightweight 1D set intersections over index arrays. We also provide a detailed comparison in the Appendix of our technical report [29].

4 A NOVEL SPARSITY ESTIMATOR: RANGER

Inspired by LayeredGraph [14], in this section we develop a novel, efficient yet accurate sparsity estimator Ranger based on CoreMat. We first provide an overview of Ranger in Section 4.1, then perform theoretical analysis in Section 4.2, and finally highlight its key advantages over prior estimators in Section 4.3.

4.1 Overview of Ranger

Recall that LayeredGraph first builds the multi-partite graph of the input SMM, and then estimates the row-wise sparsity via estimating the reachable set size in the graph by propagating the random features along the graph using min pooling. As shown in the literature [31], it achieves the highest accuracy among all the existing sparsity estimators, but its efficiency is poor due to the huge cost of multi-graph construction and feature propagation on it. To alleviate the efficiency issue, we propose to build the multi-partite graph for the CoreMat, rather than the original SMM problem, which results in a much smaller multi-partite graph, thereby improving the efficiency of sparsity estimation. We further observe that LayeredGraph uses the min pooling to estimate the reachable set size, which is inevitably biased by extreme values, leading to unstable estimation results. To tackle this problem, we

leverage the column-wise sample range of the random features for estimation, achieving more accurate and robust results.

Given an SMM problem, the key steps of Ranger are as follows:

- (1) Extract its CoreMat by Algorithm 1 and then build the multi-partite graph of the CoreMat, denoted by $H = (V^*, E^*)$.
- (2) For each node v_i^{L+1} in $V_{L+1}^* \subseteq V^*$, sample a vector $t_i \in \mathbb{R}^d$ independently from the uniform distribution $U(0, 1)$, where d is the number of samples, and then assign the node an initial feature vector

$$F(v_i^{L+1}) = (t_i \parallel -t_i), \quad (7)$$

where $\parallel$ denotes the concatenation of t_i and its opposite vector $-t_i$, and $F(v_i^{L+1}) \in \mathbb{R}^{2d}$.

- (3) Iteratively propagate the feature vectors of nodes in the $(l+1)$ -th partite to the nodes in the l -th partite, where l ranges from L to 1. Specifically, for each node $v_i^l \in V_l^* \subseteq V^*$, update its feature vector $F(v_i^l)$ by taking the column-wise maximum of the features of its connected nodes in V_{l+1} , which are denoted by a set $N(v_i^l)$

$$F(v_i^l) = \max_{v \in N(v_i^l)} F(v), \quad (8)$$

where $\max(\cdot)$ denotes the column-wise max operator.

- (4) For every node v_i^l , compute its sample range

$$r(v_i^l) = \frac{1}{d} \sum_{j=1}^d (F(v_i^l)[j] + F(v_i^l)[d+j]), \quad (9)$$

and estimate the number of nodes reached from V_{L+1}^* to it

$$\widehat{R}(v_i^l) = \frac{2}{1 - r(v_i^l)} - 1. \quad (10)$$

- (5) Derive the estimated row-wise sparsity of the i -th row of O

$$\widehat{\eta}(O, i) = \frac{\widehat{R}(v_i^1)}{|V_{L+1}|}. \quad (11)$$

EXAMPLE 6. Let Figure 9 be the multi-partite graph of an SMM problem's CoreMat and $d = 2$. First, for node v_4^1 , assume we sample a vector of 2 random variables, $(0.4, 0.6)$, concatenate with its opposite vector, and set $F(v_4^1) = (0.4, 0.6, -0.4, -0.6)$. Similarly, we initialize $F(v_4^2)$ and $F(v_4^3)$ by some vectors. Next, we update the feature vectors of v_3^1 , v_3^2 , and v_3^3 using their neighbors in V_4 . E.g., we update $F(v_3^3)$ as $(0.7, 0.3, -0.1, -0.2)$ by taking the column-wise maximum on $F(v_4^2)$ and $F(v_4^3)$. Sequentially, we update the feature vectors of nodes in V_2 and V_1 . Afterwards, we get the sample ranges $r(v_1^1) = 0$ and $r(v_1^2) = 0.5$ by Eq. (9), and estimate the reachable set sizes as $\widehat{R}(v_1^1) = 1$ and $\widehat{R}(v_1^2) = 3$ by Eq. (10). Hence, we estimate the sparsity of the first and second rows of output matrix O as $\widehat{\eta}(O, 1) = 1/3$ and $\widehat{\eta}(O, 2) = 3/3$, respectively.

Algorithm 2 presents the detailed steps of implementing Ranger.

LEMMA 5 (TIME COMPLEXITY OF ALGORITHM 2). The time complexity of Ranger is $O\left(\sum_{l=1}^L \text{nnz}(A_l) + |V_{L+1}^*| \cdot d\right)$

PROOF. Let $N_{\text{total}} = \sum_{l=1}^L \text{nnz}(A_l)$ denote the total number of non-zero elements in the input sparse matrix chain, and let M be the number of non-zero elements in the extracted CoreMat. Note that $M \leq N_{\text{total}}$.

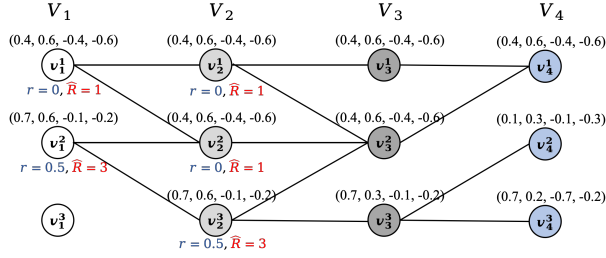


Figure 9: An example illustrating the workflow of Ranger.

Algorithm 2: Ranger

Input: A sparse matrix chain $\{\text{COO}_{A_l}\}_{l=1}^L$, an integer d

- 1 Extract CoreMat by Algorithm 1 and build its multi-partite graph $H = (V, E)$;
- 2 **for** each node $v_i^{L+1} \in V_{L+1}$ **do**
- 3 Independently sample a vector t_i of d variables from $U(0, 1)$;
- 4 $F(v_i^{L+1}) \leftarrow (t_i \| -t_i)$;
- 5 **for** $l \leftarrow L, L-1, \dots, 1$ **do**
- 6 **for** each $v_i^l \in V_l$ **do**
- 7 Compute $F(v_i^l)$ by Eq. (8);
- 8 **if** $l < L$ **then** Estimate $\widehat{R}(v_i^l)$ with $F(v_i^l)$ by Eq. (9) and (10);
- 9 **return** $\widehat{\eta}(O, i) = \frac{\widehat{R}(v_i^1)}{|V_{L+1}|}$ for every possible i ;

The overall time complexity consists of three main components:

- (1) **CoreMat extraction (Algorithm 1):** As analyzed previously, this step requires $O(N_{\text{total}})$.
- (2) **Random sampling and initialization:** For each node $v_i^{L+1} \in V_{L+1}^*$, we sample a d -dimensional vector and compute its initial feature $F(v_i^{L+1})$, which takes $O(d)$ per node. Thus, the total cost is $O(|V_{L+1}^*| \cdot d)$.
- (3) **Layer-wise propagation (Lines 5–8):** Processing all layers from $l = L$ down to 1 involves aggregating information from neighbors. Since the total number of edges in H is $O(M)$, the cumulative work across all layers is $O(M) = O(N_{\text{total}})$.

Summing all components, the total time complexity is:

$$O(N_{\text{total}}) + O(|V_{L+1}^*| \cdot d) + O(N_{\text{total}}) = O(N_{\text{total}} + |V_{L+1}^*| \cdot d) \quad (12)$$

□

4.2 Theoretical Analysis of Ranger

In this section, we first introduce the statistical estimator used in Ranger via Lemma 9, and then derive its theoretical error bound with respect to the number of samples.

LEMMA 6. *Given a random variable sequence $\{x_i\}_{i=1}^k$ independently sampled from uniform distribution $U(0, 1)$, denote $x_{(i)}$ as its order statistic, i.e., $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(k)}$. Then, the sample range $r = x_{(k)} - x_{(1)}$ follows a Beta distribution [42] with shape parameters $\alpha = k - 1$, $\beta = 2$, denoted by $\text{Be}(k - 1, 2)$.*

PROOF. See Appendix of our technical report [29]. □

THEOREM 7. *Given a random variable sequence $\{x_i\}_{i=1}^k$, where each x_i is sampled independently from $U(0, 1)$ and k is unknown, and assume we have access to $x_{\max} = \max\{x_i\}_{i=1}^k$ and $x_{\min} = \min\{x_i\}_{i=1}^k$, then an estimator of k using the sample range $r = x_{\max} - x_{\min}$ is*

$$\widehat{k} = \frac{2}{1-r} - 1 \quad (13)$$

PROOF. See Appendix of our technical report [29]. □

Next, we prove that for any fixed $\epsilon > 0$, the estimation error bound of $\widehat{k}$ with respect to the number of samples d is $O\left(\frac{1}{2d\epsilon^2}\right)$.

THEOREM 8 (ERROR BOUND). *Assume that we have d independent samples of r , and we substitute the r in Theorem 7 by $\bar{r} = \frac{1}{d} \sum_{i=1}^d r_i$, the relative error of Ranger is bounded by:*

$$\mathbb{P}(|\widehat{k} - k| \geq \epsilon k) = O\left(\frac{1}{2d\epsilon^2}\right) \quad (14)$$

where $\widehat{k}$ and k are the estimated row-wise NNZ and ground truth, respectively.

PROOF. By directly using Chebyshev's inequality and setting $\mu = \frac{k-1}{k+1}$, for any $\epsilon > 0$, we have

$$\mathbb{P}(|\bar{r} - \mu| \geq \epsilon \mu) \leq \frac{\text{Var}(\bar{r})}{(\epsilon \mu)^2} = \frac{2}{d(k-1)(k+2)\epsilon^2} \quad (15)$$

Note that

$$\widehat{k} = \frac{2}{1-\bar{r}} - 1 = \frac{1+\bar{r}}{1-\bar{r}} \quad (16)$$

Then, we have

$$\left| \frac{\widehat{k} - k}{k} \right| = \left| \frac{\frac{1+\bar{r}}{1-\bar{r}} - k}{k} \right| = \left| \frac{\bar{r} - \frac{k-1}{k+1}}{k(1-\bar{r})} \right| (k+1) \quad (17)$$

Let $\epsilon' = 2 - \frac{4}{\epsilon+2}$. Then, we claim that

$$\left| \bar{r} - \frac{k-1}{k+1} \right| \geq \frac{1}{k+1} \epsilon' \quad (18)$$

With the above claim, from Eq. (8), we have:

$$\begin{aligned} \mathbb{P}(|\widehat{k} - k| \geq \epsilon k) &\leq \mathbb{P}\left(\left| \bar{r} - \frac{k-1}{k+1} \right| \geq \frac{1}{k+1} \epsilon'\right) \leq \frac{\frac{1}{d} \frac{2(k-1)}{(k+1)^2(k+2)}}{\epsilon'^2 \left(\frac{1}{k+1}\right)^2} \\ &= \frac{1}{d} \frac{\frac{2(k-1)}{k+2}}{\frac{4\epsilon^2}{(\epsilon+2)^2}} = O\left(\frac{1}{2d\epsilon^2}\right) \end{aligned}$$

□

For the proof of the claim Eq. (18), please refer to the Appendix of our technical report [29].

We remark that although our theoretical analysis above is based on the boolean matrices, both CoreMat and Ranger can be seamlessly adapted to sparse matrices with floating point values and with properties like triangular, diagonal and symmetry, since the graph representation of sparse matrix is universally applicable.

4.3 Advantages over Prior Sparsity Estimators

In this section, we highlight the three key advantages of Ranger over prior sparsity estimators:

- *When the input sparse matrix chain becomes long, Ranger is more accurate than prior estimators.* The accuracy of prior estimators often becomes lower when the input matrix chain becomes longer [31, 46], due to the propagation of estimation error. In contrast, Ranger is able to achieve high accuracy in such cases, as the structural property of multi-partite graph and the feature propagation mechanism ensure that, every node stores the actual column-wise maximum/minimum feature of the reachable nodes in V_{L+1} .

- *Compared to LayeredGraph [14], Ranger is not only more efficient, but also more accurate.* Both LayeredGraph and Ranger use a multi-partite graph to estimate the sparsity. However, since Ranger extracts the CoreMat of the SMM problem, which is even sparser than the input matrices, the constructed multi-partite graph has fewer nodes and edges, so the multi-partite graph construction and nodes' feature propagation can be done faster. Moreover, unlike LayeredGraph [14], which relies on the sample minimum (a single order statistic), Ranger utilizes the sample range to mitigate numerical instability caused by extreme values. We also provide a detailed comparison between Ranger and LayeredGraph in the appendix of our technical report [29].

- *Ranger is more favorable for real-world sparse matrices.* Prior works [46] have shown that the sparse matrices in real applications often exhibit a pattern that most of the rows have exactly one non-zero element. For such matrices, Ranger is able to return the true sparsity even by setting $d=1$ as the sample range is always 0.

5 EXPERIMENTS

We now present the experimental results. Section 5.1 introduces the setup. We discuss the results in Sections 5.2 and 5.3.

Table 3: Statistics of datasets used in our experiments.

Dataset	Vertex count	Edge count	Number of SMM problems
FourSquare (FS)	43.2K	405.5K	(32, 80, 100, 100, 100)
IMDB (IB)	471.8K	521.0K	(18, 36, 54, 100, 100)
DBLP (DP)	490.7K	1.1M	(18, 36, 54, 100, 100)
DBpedia (DB)	9.0M	31.2M	(100, 100, 100, 100, 100)
FreeBase (FB)	29.1M	105.6M	(100, 100, 100, 100, 100)

5.1 Setup

Datasets. We use the sparse matrices from two real-world sources:

(1) **Heterogeneous Information Networks (HINs):** We adopt the experimental framework from [31] by deriving sparse matrix chains from five HINs: FourSquare¹, IMDB², DBLP³, DBpedia⁴, and FreeBase⁵. Table 3 summarizes the statistics of each HIN. HINs are selected as the source of sparse matrix chains due to their inherent correlations and the rich semantic diversity embedded in their metapaths. As multiplying a metapath of length L directly corresponds to

¹<https://sites.google.com/site/home/foursquare-dataset>

²<https://www.imdb.com/interfaces/>

³<http://dblp.uni-trier.de/xml/>

⁴<https://wiki.dbpedia.org/Datasets>

⁵<http://freebase-easy.cs.uni-freiburg.de/dump/>

a SMM problem of length L , this alignment enables a robust evaluation of CoreMat and Ranger. For each dataset, metapaths of lengths [2, 3, 4, 5, 6, 7] were generated (Table 3). Specifically, we sampled 100 metapaths per length in cases with over 100 valid metapaths (those encoding meaningful relationships) to ensure computational feasibility while preserving diversity. A more detailed introduction of the metapaths can be found in the appendix of our technical report [29].

(2) **Scientific Computing.** We use 14 sparse matrices from the SuiteSparse Matrix Collection [28], which includes various matrices from scientific computing, e.g., optimization problems and directed graphs. We show the matrix name, dimensions (number of rows and cols), number of non-zeros and their belonging scientific areas in the appendix of our technical report [29]. For all 14 sparse matrices, we enumerate all the possible matrix chains $A_1 \in \mathbb{R}^{m_1 \times n_1}, A_2 \in \mathbb{R}^{m_2 \times n_2}, A_3 \in \mathbb{R}^{m_3 \times n_3}$ of length 3, containing 2, 184 chains, and compared Ranger with the baselines on these chains. For unmatched dimensions, we trim the upper left square matrix of shape (l, l) from each input matrix where:

$$l = \min\{\min_{i \in \{1,2,3\}}(m_i), \min_{i \in \{1,2,3\}}(n_i)\}.$$

Note that different sparse estimators favor different sparse matrix formats. We simply use COO format for input matrices since COO can be easily transformed to any other format.

Sparsity estimators. We test the following sparsity estimators:

- MetaAC [8] is a naive average case sparsity estimator;
- DensityMap (DM) [26] is a sparsity estimator that splits the matrices into blocks of size b , we set $b = 256$ by default;
- MNC [46] is a counting based sparsity estimator that leverages structural information;
- LayeredGraph (LG) [14] is a graph based sparsity estimator that achieves SOTA accuracy, we set $d = 16$ by default unless otherwise stated;
- RS-estimator (RSE) [31] is a row-wise sparsity estimator that retains accuracy with considerable efficiency;
- Ranger: Our proposed sparsity estimator, default $d = 16$.

All the sparsity estimators mentioned above are implemented in C++ and compiled with the gcc 9.4.0 compiler using the -O3 optimization level. The experiments are run on a Linux machine running Ubuntu Linux 20.04.5 LTS, equipped with dual Intel Xeon(R) Gold 6338 2.0GHz processors (64 cores) and 496GB of RAM. We set the number of threads = 16 by default.

We evaluate the accuracy of sparsity estimators by the metric relative error following [31, 46], which is defined as follows.

$$\varepsilon(O, \widehat{O}) = \frac{\max\{\rho(O), \rho(\widehat{O})\}}{\min\{\rho(O), \rho(\widehat{O})\}} \quad (19)$$

5.2 Evaluation on CoreMat

We evaluate the effect of CoreMat on SMM (we simply set $L = 3$).

- **Reduction of matrix size and multiplications.** Figure 10(a) shows the total NNZ in the original SMM and its CoreMat. Clearly, the size of the matrices in the original matrices could be up to 500 times larger than that of CoreMat. Besides, Figure 10(b) compares the total number of multiplications when executing the original SMM and its CoreMat. We see that the total number of multiplications in CoreMat is much smaller, since many non-zero elements have been removed.

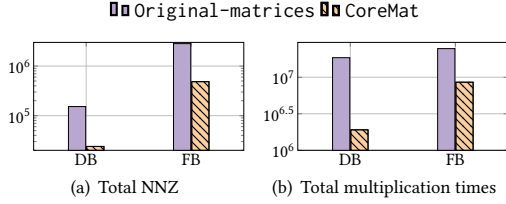


Figure 10: Total NNZ and multiplication times.

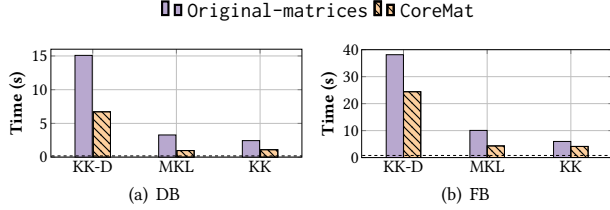


Figure 11: Effect of CoreMat on existing SMM methods.

• **Improving existing SMM methods by using CoreMat.** This experiment aims to show that for any existing SMM method, it is possible to improve its efficiency by first deriving the CoreMat and then executing the method on CoreMat. Specifically, we consider three existing SMM methods, i.e., MKL [23], Kokkos-DEFAULT (KK-D), and Kokkos (KK) [40], and for each of them, we create a variant which first extracts the CoreMat and runs it on the CoreMat. Figure 11 reports the efficiency of these methods on DB and FB datasets. Each bar shows the end to end time to finish the SMM, including the time to construct CoreMat, which is indicated by the gray dashed line on each figure. Obviously, by using CoreMat, the efficiency of all SMM methods can be improved. Hence, CoreMat can serve as a plug-in method for improving the efficiency of all the existing SMM methods.

5.3 Evaluation on Ranger

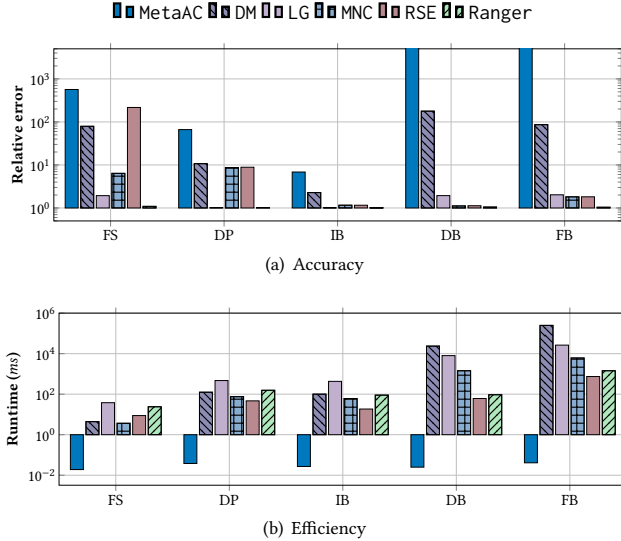


Figure 12: Comparison of sparsity estimators ($L=2$).

• **Sparsity estimation ($L = 2$).** In this experiment, we use the sampled matrix pairs from five datasets as input matrices and obtain ground-truth NNZ by executing exact SMM. Figure 12(a) reports the relative error. Clearly, Ranger achieves the highest accuracy

three of the five datasets, and comparable accuracy with LG on the remaining two datasets. The relative error of MetaAC, DM, MNC and RSE exhibit drastic oscillation among different datasets, because the underlying independence and uniformity assumption they adopted do not hold in practice.

Figure 12(b) presents the efficiency results. We see that MetaAC is the fastest one, since it simply uses the overall sparsity of all the input matrices for estimation. RSE is the second fastest one since it uses the sparsity values of each row of the input matrices, which can be computed efficiently. However, although these two methods achieve the best efficiency, their accuracy is poor, making them inapplicable for real-world applications. The remaining three estimators, i.e., DM, LG, and MNC, not only achieve higher relative errors but also lower efficiency than Ranger. Hence, Ranger achieves the best trade-off between accuracy and efficiency, especially when the matrix dimensions are large. For example, on million-scale datasets like DB and FB, Ranger has comparable efficiency with SOTA methods like MNC and RSE, but achieves much higher accuracy.

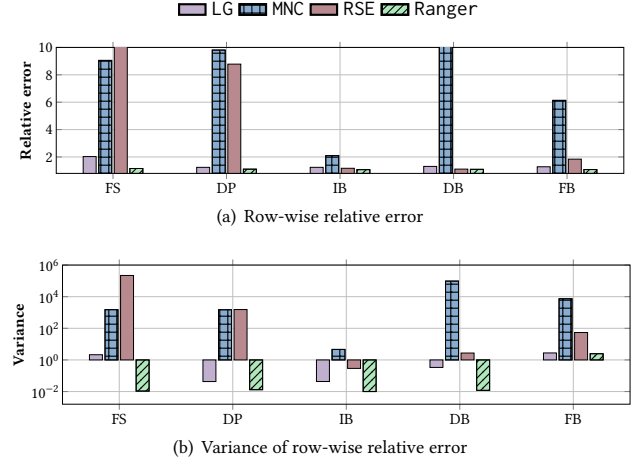


Figure 13: Comparison of row-wise sparsity estimators.

Additionally, we report the accuracy of the row-wise sparsity estimation for the four estimators that can provide row-level estimation. For any i such that $nnz(O[i, :]) > 0$, its row-wise relative error is defined as:

$$\varepsilon(i) = \frac{\max\{\eta(O[i, :]), \widehat{\eta(O[i, :])}\}}{\min\{\eta(O[i, :]), \widehat{\eta(O[i, :])}\}} \quad (20)$$

Figure 13(a) shows that Ranger and LG achieve consistently small row-wise relative error while the accuracy of RSE and MNC vary among datasets. Clearly, Ranger's row-wise relative error is the smallest among the competing methods on all the five datasets. Furthermore, we present the variance of the relative error in Figure 13(b), where Ranger achieves the smallest values under all settings. RSE and MNC endure drastic oscillation, with numbers as large as 10^4 , while the corresponding results of Ranger and LG are consistently low. Thus, Ranger robustly outperforms existing sparsity estimators in row-wise accuracy with comparable efficiency.

• **Sparsity estimation ($L \geq 3$).** We consider the scenario when multiplying three or more matrices. We report the relative error and runtime on HINs in Figures 14 and 15 respectively, where the x-axis indicates the number of input matrices. Obviously, Ranger

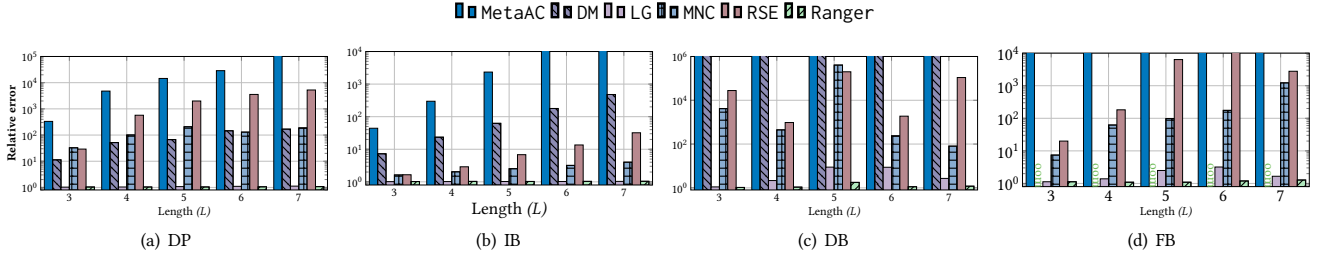


Figure 14: Accuracy of sparsity estimators for SMM with $L \geq 3$.

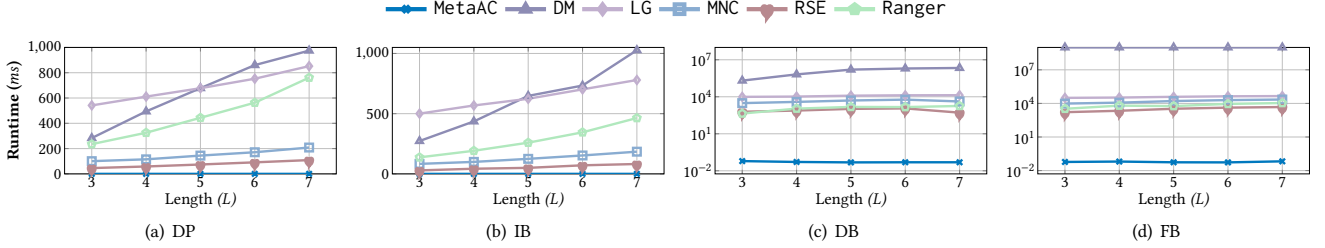


Figure 15: Runtime of sparsity estimators for SMM with $L \geq 3$.

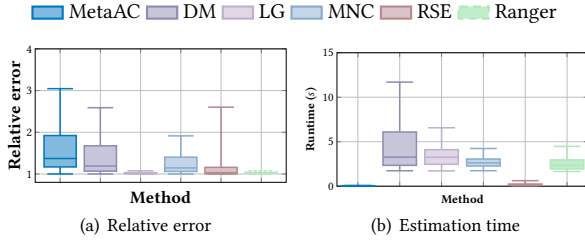


Figure 16: Performance comparison on SuiteSparse [28].

achieves the best or comparable accuracy on all datasets with arbitrary length, and for large matrices DB and FB, it costs comparable runtime with MNC and RSE but attains much higher accuracy. Besides, sparsity estimators other than LG and Ranger suffer from error propagation, i.e., as the number of matrices increases, their relative error increases exponentially [46], because they use the estimated sparsity of $A_{l+1:L}$ to predict $\rho(A_{l:L})$. As for LG and Ranger, they demonstrate little error propagation, and Ranger outperforms LG on FB and DB in terms of accuracy and is consistently faster than LG under all settings.

For experiments on scientific computing matrices, we report the accuracy and execution time as box-plots in Figure 16, displaying the corresponding median, lower quartile, upper quartile, minimum, and maximum values. As we can see from Figure 16(a), Ranger and LG [14] achieve the best accuracy and robustness, while MNC [46], DensityMap [26] and RSE [31] have higher and comparative median relative error. In the sense of efficiency, we can see from Figure 16(b) that, MetaAC and Rose are faster than Ranger, but their relative errors are much larger.

• **Effect of the number of samples d .** For sparsity estimators based on multi-partite graphs, the hyperparameter d has a significant effect on the performance. We compare Ranger with LG and LG-Uni [14], which is a variant of LG, using uniform distribution instead of exponential distribution to sample the initial features. Figures 17 and 18 report the effect of d on DB and FB respectively, where $d \in [2, 4, 8, 16, 32, 64]$, with the results averaged over 5 seeds. Clearly, Ranger outperforms LG and LG-Uni in both accuracy and

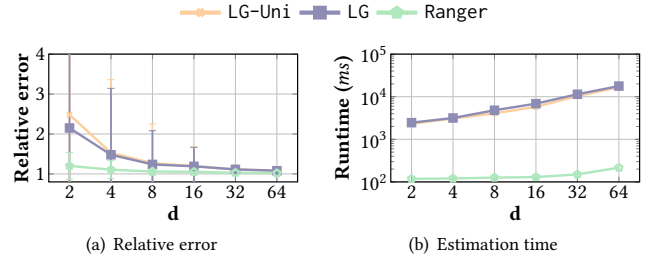


Figure 17: Effect of d on accuracy and efficiency on DB.

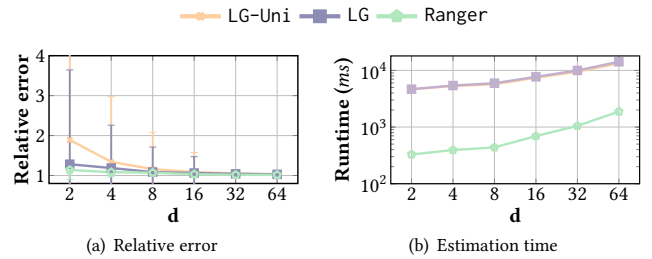


Figure 18: Effect of d on accuracy and efficiency on FB.

efficiency for $d \in [2, 4, 8, 16]$, and have smaller variance. The reason is that, LG and LG-Uni rely on a single order statistic which might cause unstable estimation, since minimum, as an extreme value, can have drastic oscillations, especially when d is small. As the vector size d increases, all three methods achieve a decreasing relative error, but consume more time cost. In addition, LG takes slightly higher time cost than LG-Uni since sampling from exponential distribution is more costly than that from uniform distribution.

• **Overall SMM efficiency.** This experiment aims to compare the overall efficiency of different SMM methods with $L \geq 3$. Specifically, we first select MKL as a backbone SMM method, and then combine it with five different sparsity estimators to execute the SMM respectively. For each sparsity estimator, we first use it to estimate the sparsity of all sub-chains for a given SMM problem, then

use the calibrated cost model Eq. (6) to get the optimal parentheses, and finally execute the SMM by using MKL accordingly.

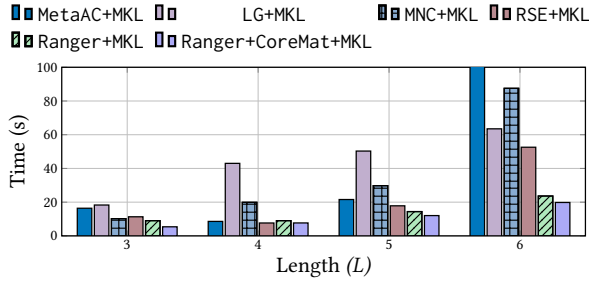


Figure 19: Overall efficiency evaluation of SMM on FB.

Figure 19 presents the efficiency of SMM. We observe that both Ranger+MKL and CoreMat+Ranger+MKL are faster than all the other methods, and CoreMat+Ranger+MKL achieves the best efficiency. For example, when $L = 6$, CoreMat+Ranger+MKL is 2.5 times faster than the SOTA method. By jointly examining the sparsity estimation and end to end time cost, though methods like MetaAC, MNC and RSE take comparably small estimation time, but their estimation accuracy are insufficient to determine the optimal parentheses, so they undertake larger latency in the SMM execution period. LG has comparable accuracy with Ranger, but it takes too much time for estimation and further dampen the overall latency. Hence, our newly proposed CoreMat and Ranger are effective for improving the efficiency of existing SMM methods.

6 RELATED WORK

In this section, we mainly review existing works on SMM methods, sparsity estimators, and acyclic join processing.

- **SMM methods.** Most of the SMM methods follow the Gustavson’s algorithm [21], which enables efficient parallelism [15, 20, 23, 32, 34, 36, 40, 45]. These works often incorporate a sparsity estimation stage before numeric multiplication. For example, MKL [23], cuSPARSE [36] and Kokkos [40] use the precise prediction in this stage to exploit the hardware capacity, but take non-trivial computing overheads, because the precise prediction method essentially multiplies the boolean input matrices in non-linear time [1, 3, 26, 56]. Although the outer-product based SMM [2, 10, 11, 37, 56] can circumvent such redundancy when $L = 2$, it is not applicable as L increases. Several works have studied SMM with $L > 2$, in which redundant computation is ubiquitous, requiring an optimal parentheses before numeric multiplication. To find the optimal parentheses, Chikalov et al. [12] optimize sequential order using various cost functions; Myunga et al. [35] implement a MapReduce-based solution using database-style joins; and Lin et al. [31] utilize a row-wise sparsity estimator to determine the parentheses. Nevertheless, even with exact row-wise sparsity estimation, the above works cannot avoid the multiplications between non-zero and zero elements, leading to much redundant computation.

- **Sparsity estimators.** As aforementioned, there are three groups of prior sparsity estimators, i.e., probability-, sampling-, and graph-based methods. The probability-based methods [26, 31, 46] essentially apply the inclusion-exclusion principle (IEP) [7] at different scales. MetaAC [8] uses the overall sparsity of input matrices to estimate the sparsity of output matrix, corresponding to the average case estimation. Sparso [41] further refines MetaAC and

makes accommodations to symmetric, triangular and diagonal matrices. DensityMap [26] partitions the matrix into some blocks and utilizes the block-wise sparsity to estimate the overall sparsity. MNC [46] first builds a novel matrix sketch, then uses the rows/columns with only one non-zero element in the input matrices to derive the exactly known non-zero elements of the output matrix, while the remaining NNZ is predicted with generic estimators like DensityMap. Rose [31] estimates the row-wise sparsity of $O[i, :]$, by iterating the non-zero elements of $A[i, :]$ and summing up the sparsity of the corresponding rows in B . The above sparsity estimators make assumptions on the uniform distribution of non-zero entries at matrix-level [8], block-level [26] and row-level [31], which introduces inductive bias into their estimation.

For the sampling-based estimators [4, 5, 55], they sample the rows/columns from matrix A and B , respectively, and derive the sampled outer products. However, they either cannot converge to the true sparsity, or cause inexpensive computation overhead. Additionally, they are inapplicable when dealing with SMM problems with $L > 2$, since it requires the input matrices to be readily available for sampling. The graph-based method LayeredGraph [14] transforms the row-wise sparsity estimation problem to estimating the size of reachable set, but constructing and propagating on the full graph representation of SMM takes huge time overhead.

- **Acyclic join query.** In relational databases, a join query is called an acyclic join query if its associated hypergraph satisfies the property of acyclicity. To process acyclic join, people often apply the Robust Query Processing (RQP) [6] techniques to address the cardinality explosion issue, and a well-known algorithm is the Yannakakis algorithm [53] which uses two-pass reduction to remove the dangling tuples in the join process. As aforementioned, SMM and acyclic join processing can be unified under the semiring framework [44], since SMM operates on the Arithmetic Semiring $(\mathbb{R}, +, \times)$, while the acyclic join operates on the Boolean Semiring $(\{0, 1\}, \vee, \wedge)$ [9]. As a result, the RQP algorithms can be applied for computing the CoreMat for SMM. However, directly applying conventional RQP techniques to SMM structural pruning exhibits severe limitations: traditional RQP heavily relies on dynamic relational runtime adaptations, adaptive re-optimization, or query plan hedging [49, 52]. When translated to numerical sparse linear algebra, these database-level mechanisms introduce costly relational overheads.

7 CONCLUSIONS

To enable efficient SMM, in this paper we introduce a novel concept CoreMat, which significantly sparsifies the input matrices so that the multiplications between zero elements and non-zero elements can be completely avoided, while ensuring the correctness of the output matrix. Based on CoreMat, we further develop Ranger, a novel efficient yet accurate sparsity estimator with theoretical guarantee. Extensive experimental results demonstrate that CoreMat is effective for sparsifying the input matrices, and Ranger significantly outperforms existing estimators in accuracy while achieving comparable efficiency with the most efficient sparsity estimators. In the future, we will implement our methods in the distributed computation platforms (e.g., Spark), enhance the parallelization using core decomposition techniques and also test their performance in more real-world application scenarios.

REFERENCES

- [1] Amir Abboud, Karl Bringmann, Nick Fischer, and Marvin Künnemann. 2023. The Time Complexity of Fully Sparse Matrix Multiplication. arXiv:2309.06317 [cs.DS] <https://arxiv.org/abs/2309.06317>
- [2] Kadir Akbudak and Cevdet Aykanat. 2014. Simultaneous Input and Output Matrix Partitioning for Outer-Product-Parallel Sparse Matrix-Matrix Multiplication. *SIAM Journal on Scientific Computing* 36, 5 (2014), C568–C590. <https://doi.org/10.1137/13092589X> arXiv:<https://doi.org/10.1137/13092589X>
- [3] Josh Alman. 2019. Limits on the Universal Method for Matrix Multiplication. arXiv:1812.08731 [cs.CC] <https://arxiv.org/abs/1812.08731>
- [4] Rasmus Resen Amossen, Andrea Campagna, and Rasmus Pagh. 2011. Better size estimation for sparse matrix products. arXiv:1006.4173 [cs.DS] <https://arxiv.org/abs/1006.4173>
- [5] Pham Nguyen Quang Anh, Rui Fan, and Yonggang Wen. 2016. Balanced Hashing and Efficient GPU Sparse General Matrix-Matrix Multiplication. In *Proceedings of the 2016 International Conference on Supercomputing (Istanbul, Turkey) (ICS '16)*. Association for Computing Machinery, New York, NY, USA, Article 36, 12 pages. <https://doi.org/10.1145/2925426.2926273>
- [6] Altan Birlir, Alfons Kemper, and Thomas Neumann. 2024. Robust Join Processing with Diamond Hardened Joins. *Proc. VLDB Endow.* 17 (2024), 3215–3228.
- [7] Andreas Björklund, Thore Husfeldt, and Mikko Koivisto. 2009. Set Partitioning via Inclusion-Exclusion. *SIAM J. Comput.* 39, 2 (2009), 546–563. <https://doi.org/10.1137/070683933> arXiv:<https://doi.org/10.1137/070683933>
- [8] Matthias Boehm, Douglas Burdick, Alexandre V. Evfimievski, Berthold Reinwald, Frederick Reiss, Prithviraj Sen, Shirish Tatikonda, and Yuanyuan Tian. 2014. SystemML's Optimizer: Plan Generation for Large-Scale Machine Learning Programs. *IEEE Data Eng. Bull.* 37 (2014), 52–62. <https://api.semanticscholar.org/CorpusID:10001564>
- [9] Matthias Boehm, Matteo Interlandi, and Chris Jermaine. 2023. Optimizing Tensor Computations: From Applications to Compilation and Runtime Techniques. In *Companion of the 2023 International Conference on Management of Data (Seattle, WA, USA) (SIGMOD '23)*. Association for Computing Machinery, New York, NY, USA, 53–59. <https://doi.org/10.1145/3555041.3589407>
- [10] Aydin Buluc and John R. Gilbert. 2008. Challenges and Advances in Parallel Sparse Matrix-Matrix Multiplication. In *2008 37th International Conference on Parallel Processing*, 503–510. <https://doi.org/10.1109/ICPP.2008.45>
- [11] Aydin Buluc and John R. Gilbert. 2008. On the representation and multiplication of hypersparse matrices. In *2008 IEEE International Symposium on Parallel and Distributed Processing*, 1–11. <https://doi.org/10.1109/IPDPS.2008.4536313>
- [12] Igor Chikalov, Shahid Hussain, and Mikhail Moshkov. 2011. Sequential optimization of matrix chain multiplication relative to different cost functions. In *Proceedings of the 37th International Conference on Current Trends in Theory and Practice of Computer Science (Nový Smokovec, Slovakia) (SOFSEM '11)*. Springer-Verlag, Berlin, Heidelberg, 157–165.
- [13] E. Cohen. 1994. Estimating the size of the transitive closure in linear time. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 190–200. <https://doi.org/10.1109/SFCS.1994.365694>
- [14] Edith Cohen. 1998. Structure Prediction and Computation of Sparse Matrix Products. *Journal of Combinatorial Optimization* 2 (1998), 307–332. <https://api.semanticscholar.org/CorpusID:139115>
- [15] Steven Dalton, Luke Olson, and Nathan Bell. 2015. Optimizing Sparse Matrix-Matrix Multiplication for the GPU. *ACM Trans. Math. Softw.* 41, 4, Article 25 (Oct. 2015), 20 pages. <https://doi.org/10.1145/2699470>
- [16] Zhaoyang Du, Yijin Guan, Tianchan Guan, Dimin Niu, Nianxiong Tan, Xiaopeng Yu, Hongzhong Zheng, Jianyi Meng, Xiaolang Yan, and Yuan Xie. 2023. Predicting the Output Structure of Sparse Matrix Multiplication with Sampled Compression Ratio. In *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*, 483–490. <https://doi.org/10.1109/ICPADS56603.2022.00069>
- [17] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding. In *Proceedings of The Web Conference 2020 (WWW '20)*. ACM. <https://doi.org/10.1145/3366423.3380297>
- [18] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-matrix Multiplication. *ACM Comput. Surv.* 55, 12, Article 244 (March 2023), 36 pages. <https://doi.org/10.1145/3571157>
- [19] Christos Giatsidis, Dimitrios M. Thilikos, and Michalis Vazirgiannis. 2011. D-cores: Measuring Collaboration of Directed Graphs Based on Degeneracy. In *2011 IEEE 11th International Conference on Data Mining*, 201–210. <https://doi.org/10.1109/ICDM.2011.46>
- [20] Felix Gremse, Andreas Höfer, Lars Ole Schwen, Fabian Kiessling, and Uwe Nannmann. 2015. GPU-Accelerated Sparse Matrix-Matrix Multiplication by Iterative Row Merging. *SIAM Journal on Scientific Computing* 37, 1 (2015), C54–C71. <https://doi.org/10.1137/130948811> arXiv:<https://doi.org/10.1137/130948811>
- [21] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (Sept. 1978), 250–269. <https://doi.org/10.1145/355791.355796>
- [22] William L. Hamilton, Rex Ying, and Jure Leskovec. 2018. Inductive Representation Learning on Large Graphs. arXiv:1706.02216 [cs.SI] <https://arxiv.org/abs/1706.02216>
- [23] Intel. 2025. *Intel Math Kernel Library*. <https://software.intel.com/en-us/mkl>
- [24] Akshay Java, Xiaodan Song, Tim Finin, and Belle Tseng. 2007. Why we twitter: understanding microblogging usage and communities. In *Proceedings of the 9th WebKDD and 1st SNA-KDD 2007 Workshop on Web Mining and Social Network Analysis (San Jose, California) (WebKDD/SNA-KDD '07)*. Association for Computing Machinery, New York, NY, USA, 56–65. <https://doi.org/10.1145/1348549.1348556>
- [25] Humayun Kabir and Kamesh Madduri. 2017. Parallel k-Core Decomposition on Multicore Platforms. In *2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 1482–1491.
- [26] David Kernert, Frank Köhler, and Wolfgang Lehner. 2015. SpMacho - Optimizing Sparse Linear Algebra Expressions with Probabilistic Density Estimation. In *International Conference on Extending Database Technology*. <https://api.semanticscholar.org/CorpusID:16552471>
- [27] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. arXiv:1609.02907 [cs.LG] <https://arxiv.org/abs/1609.02907>
- [28] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* 4, 35 (2019), 1244. <https://doi.org/10.21105/joss.01244>
- [29] Guanghao Liang and Yixiang Fang. 2026. Structure Sparsification and Sparsity Estimation for Efficient Sparse Matrix Multiplication. Retrieved May 19, 2026 from https://github.com/GuanghaoLiang/Ranger_2026
- [30] Xuankun Liao, Qing Liu, Jiaxin Jiang, Xin Huang, Jianliang Xu, and Byron Choi. 2022. Distributed D-core Decomposition over Large Directed Graphs. arXiv:2202.05990 [cs.DB] <https://arxiv.org/abs/2202.05990>
- [31] Chunxu Lin, Wensheng Luo, Yixiang Fang, Chenhao Ma, Xilin Liu, and Yuchi Ma. 2024. On Efficient Large Sparse Matrix Chain Multiplication. *Proc. ACM Manag. Data* 2, 3, Article 156 (May 2024), 27 pages. <https://doi.org/10.1145/3654959>
- [32] Weifeng Liu and Brian Vinter. 2014. An Efficient GPU General Sparse Matrix-Matrix Multiplication for Irregular Data. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium*, 370–381. <https://doi.org/10.1109/IPDPS.2014.47>
- [33] Wensheng Luo, Yixiang Fang, Chunxu Lin, and Yingli Zhou. 2024. Efficient Parallel D-Core Decomposition at Scale. *Proc. VLDB Endow.* 17, 10 (June 2024), 2654–2667. <https://doi.org/10.14778/3675034.3675054>
- [34] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse matrix-matrix multiplication on modern architectures. In *2012 19th International Conference on High Performance Computing*, 1–10. <https://doi.org/10.1109/HiPC.2012.6507483>
- [35] Jaesook Myung and Sang-goo Lee. 2012. Matrix chain multiplication via multi-way join algorithms in MapReduce. In *Proceedings of the 6th International Conference on Ubiquitous Information Management and Communication (Kuala Lumpur, Malaysia) (ICUIMC '12)*. Association for Computing Machinery, New York, NY, USA, Article 53, 5 pages. <https://doi.org/10.1145/2184751.2184817>
- [36] NVIDIA. 2025. *cuSPARSE: CUDA Sparse Matrix Library*. <https://docs.nvidia.com/cuda/cusparse/>
- [37] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 724–736. <https://doi.org/10.1109/HPCA.2018.00067>
- [38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703 [cs.LG] <https://arxiv.org/abs/1912.01703>
- [39] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. ACM, 1981–1984.
- [40] Sivasankaran Rajamanickam, Seher Acer, Luc Berger-Vergiat, Vinh Dang, Nathan Ellingwood, Evan Harvey, Brian Kelley, Christian R. Trott, Jeremiah Wilke, and Ichitaro Yamazaki. 2021. Kokkos Kernels: Performance Portable Sparse/Dense Linear Algebra and Graph Kernels. arXiv:2103.11991 [cs.MS] <https://arxiv.org/abs/2103.11991>
- [41] Hongbo Rong, Jongsoo Park, Lingxiang Xiang, Todd A. Anderson, and Mikhail Smelyanskiy. 2016. Sparsio: Context-driven optimizations of sparse linear algebra. In *2016 International Conference on Parallel Architecture and Compilation Techniques (PACT)*, 247–259. <https://doi.org/10.1145/2967938.2967943>
- [42] Colin Rose and Murray D. Smith. 2002. mathStatica: Mathematical Statistics with Mathematica. In *Compstat, Wolfgang Härdle and Bernd Rönz (Eds.)*. Physica-Verlag HD, Heidelberg, 437–442.
- [43] Emanuele Rossi, Bertrand Charpentier, Francesco Di Giovanni, Fabrizio Frasca, Stephan Günnemann, and Michael Bronstein. 2023. Edge Directionality Improves Learning on Heterophilic Graphs. arXiv:2305.10498 [cs.LG] <https://arxiv.org/abs/2305.10498>
- [44] Louis Halle Rowen. 2026. Semirings. arXiv:2602.19209 [math.RA] <https://arxiv.org/abs/2602.19209>

- [45] Jakob Siegel, Oreste Villa, Sriram Krishnamoorthy, Antonino Tumeo, and Xiaoming Li. 2010. Efficient sparse matrix-matrix multiplication on heterogeneous high performance systems. In *2010 IEEE International Conference On Cluster Computing Workshops and Posters (CLUSTER WORKSHOPS)*. 1–8. <https://doi.org/10.1109/CLUSTERWKSP.2010.5613109>
- [46] Johanna Sommer, Matthias Boehm, Alexandre V. Evfimievski, Berthold Reinwald, and Peter J. Haas. 2019. MNC: Structure-Exploiting Sparsity Estimation for Matrix Expressions. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1607–1623. <https://doi.org/10.1145/3299869.3319854>
- [47] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonese, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 766–780. <https://doi.org/10.1109/MICRO50266.2020.00068>
- [48] Richard P. Stanley. 2015. *Catalan Numbers*. Cambridge University Press.
- [49] Mihail Stoian, Andreas Zimmerer, Skander Krid, Amadou Latyr Ngom, Jialin Ding, Tim Kraska, and Andreas Kipf. 2025. Parachute: Single-Pass Bi-Directional Information Passing. arXiv:2506.13670 [cs.DB] <https://arxiv.org/abs/2506.13670>
- [50] Yuke Wang, Boyuan Feng, Zheng Wang, Guyue Huang, and Yufei Ding. 2023. TC-GNN: Bridging Sparse GNN Computation and Dense Tensor Cores on GPUs. arXiv:2112.02052 [cs.LG] <https://arxiv.org/abs/2112.02052>
- [51] Felix Wu, Tianyi Zhang, Amauri Holanda de Souza Jr., Christopher Fifty, Tao Yu, and Kilian Q. Weinberger. 2019. Simplifying Graph Convolutional Networks. arXiv:1902.07153 [cs.LG] <https://arxiv.org/abs/1902.07153>
- [52] Yifei Yang, Hangdong Zhao, Xiangyao Yu, and Paraschos Koutris. 2025. Predicate Transfer: Efficient Pre-Filtering on Multi-Join Queries. arXiv:2307.15255 [cs.DB] <https://arxiv.org/abs/2307.15255>
- [53] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7 (Cannes, France) (VLDB '81)*. VLDB Endowment, 82–94.
- [54] Dezhong Yao, Sifan Zhao, Tongtong Liu, Gang Wu, and Hai Jin. 2024. Ap-SpGEMM: Accelerating Large-scale SpGEMM with Heterogeneous Collaboration and Adaptive Panel. *ACM Trans. Archit. Code Optim.* (Nov. 2024). <https://doi.org/10.1145/3703352> Just Accepted.
- [55] Yongyang Yu, Mingjie Tang, Walid G. Aref, Qutaibah M. Malluhi, Mostafa M. Abbas, and Mourad Ouzzani. 2017. In-Memory Distributed Matrix Computation Processing and Optimization. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. 1047–1058. <https://doi.org/10.1109/ICDE.2017.150>
- [56] Raphael Yuster and Uri Zwick. 2005. Fast sparse matrix multiplication. *ACM Trans. Algorithms* 1, 1 (July 2005), 2–13. <https://doi.org/10.1145/1077464.1077466>

8 APPENDIX

8.1 Theoretical Comparison between Ranger and LayeredGraph

In this section, we perform an-depth theoretical analysis of Ranger and its competing methods: uniform distribution based [13] and exponential distribution based [14] LayeredGraph, which are denoted by LG-Uni and LG, respectively. Our analysis shows that Ranger achieves half the estimation variance of both LG and LG-Uni, while its relative error is just 1/16 that of LG-Uni and comparable to LG when d is small. We compare the design and key theoretical results of these methods in Table 4.

Specifically, we have proved that Ranger provides a more stable prediction in the sense of estimation variance, which aligns with the intuition that Ranger leverages more information from both sample minimum and maximum than a single order statistic.

Essentially, all these three methods leverage the order statistics to estimate the sample volume. We restate the problem definition and necessary notations.

Definition 8.1. Given a series of random variables $\{x_i\}_{i=1}^N$ sampled independently from a given distribution $\mathcal{D}$, where its sample volume N is unknown, estimate N .

Table 4: Comparison of existing sparsity estimators.

Method	$\mathcal{D}$	Estimator	Relative Error	Relative Variance
LG-Uni [13]	U[0, 1]	sample minimum	$16/(\epsilon^2 d)$	$1/d$
LG [14]	Exp(1)	sample minimum	$1/\exp(\epsilon^2 d)$	$1/d$
Ranger(ours)	U[0, 1]	sample range	$1/(2\epsilon^2 d)$	$1/2d$

★ Note: the relative errors are reported and collected from each work.

In the following, we provide a variational analysis of each method.

• LG-Uni: The sample volume is estimated by $\hat{N} = g(\bar{c}) := 1/\bar{c} - 1$, where $\bar{c} = \frac{1}{d} \sum_{j=1}^d x_{(1)}^j$, $x_{(1)}^j$ denotes the sample minimum of the j -th experiment. Since $x_{(1)}^j$ follows the Beta distribution $Beta(1, N)$, the variance of $\bar{c}$ is

$$Var(\bar{c}) = \frac{k}{d(N+1)^2(k+2)} \quad (21)$$

Let $\mu = \mathbb{E}[\bar{c}] = \frac{1}{N+1}$, we uses the Taylor's expansion to analyze its variance:

$$Var(\hat{N}) \approx [g'(\mu)]^2 Var(\bar{c}) = (-(N+1)^2)^2 \frac{N}{d(N+1)^2(N+2)} \quad (22)$$

$$= \frac{N(N+1)^2}{d(N+2)} \approx \frac{N^2}{d} \quad (23)$$

Therefore, its **relative variance** is $Var(\hat{N})/N^2 = 1/d$.

• LG: The sample volume is estimated by $\hat{N} = g(s) := \frac{d-1}{s}$, where $s = \sum_{j=1}^d x_{(1)}^j$, $x_{(1)}^j$ denotes the sample minimum of the j -th experiment. $x_{(1)}^j$ follows the Gamma distribution $Gamma(d, N)$. Its first and second moments are:

$$\mathbb{E}[\hat{N}] = \mathbb{E}\left[\frac{d-1}{s}\right] = (d-1) \int_0^\infty \frac{1}{z} \frac{N^d}{\Gamma(d)} z^{d-1} e^{-Nz} dz = N \quad (24)$$

$$\mathbb{E}[\hat{N}^2] = \mathbb{E}\left[\frac{(d-1)^2}{s^2}\right] = (d-1)^2 \int_0^\infty \frac{1}{z^2} \frac{N^d}{\Gamma(d)} z^{d-1} e^{-Nz} dz \quad (25)$$

$$= \frac{d-1}{d-2} N^2 (d > 2) \quad (26)$$

Its variance $Var(\hat{N})$:

$$Var(\hat{N}) = \mathbb{E}[\hat{N}^2] - (\mathbb{E}[\hat{N}])^2 = \frac{1}{d-2} N^2 \quad (27)$$

Therefore, its **relative variance** is $Var(\hat{N})/N^2 = 1/(d-2)$.

• Ranger: The sample volume is estimated by $\hat{N} = g(\bar{r}) := \frac{2}{1-\bar{r}}$, where $\bar{r} = \frac{1}{d} \sum_{j=1}^d r$, $r = x_{(N)}^j - x_{(0)}^j$ denotes the sample range of the j -th experiment. $\bar{r}$'s variance is:

$$Var[\bar{r}] = \frac{2(N-1)}{d(N+1)^2(N+2)} \quad (28)$$

Let $\mu = \mathbb{E}[\bar{r}] = \frac{N-1}{N+1}$, we uses the Taylor's expansion to analyze its variance:

$$Var(\hat{N}) \approx [g'(\mu)]^2 Var(\bar{r}) = \left(\frac{(N+1)^2}{2}\right)^2 \frac{2(N-1)}{d(N+1)^2(N+2)} \quad (29)$$

$$= \frac{(N-1)(N+1)^2}{2d(N+2)} \approx \frac{N^2}{2d} \quad (30)$$

Therefore, its **relative variance** is $Var(\hat{N})/N^2 = 1/2d$.

With the variation analysis above, we conclude that although Ranger has comparable theoretical relative error comparing to LG, by leveraging the information of both sample maximum and minimum, it has a smaller estimation variance, which is half of that of LG and LG-Uni, thereby achieving more robust estimation.

8.2 HIN Datasets

In this section, we provide a detailed description of the use of metapaths in HINs in our experiments. To evaluate the proposed algorithms under realistic workloads, we construct an experimental framework based on real-world SMM scenarios. Following the established experimental methodologies [31], we derived various chains of sparse matrices from five diverse, real-world Heterogeneous Information Networks (HINs): FourSquare⁶, IMDB⁷, DBLP⁸, DBpedia⁹, and FreeBase¹⁰. The motivation for utilizing HINs lies in their inherent architectural diversity. Unlike homogeneous networks, HINs encompass multiple distinct types of vertices and complex multi-relational links, which yield highly diverse matrix properties and complex correlation patterns, allowing for a comprehensive evaluation of the generalization capabilities, scalability, and efficiency of the proposed algorithms.

Take the Freebase network as an example. It contains records featuring entity types such as Actors (A), Movies (M), Directors (D), and Genres (G). The link relationships between actors and movies form a bipartite graph, represented by the adjacency matrix $\mathbf{M}(AM)$. By considering a chain of such relationships—known as a meta-paths, we can extract semantic relationships between distant vertices by multiplying a chain of matrices. For instance, the meta-path $A \rightarrow M \rightarrow D \rightarrow M \rightarrow A$ maps pairs of actors who have starred in movies shared by the same director. We can find the complete set of these actor pairs by computing the product of the corresponding sparse matrix chain:

$$\mathbf{M}(AM) \times \mathbf{M}(MD) \times \mathbf{M}(DM) \times \mathbf{M}(MA) \quad (31)$$

⁶<https://sites.google.com/site/home/foursquare-dataset>

⁷<https://www.imdb.com/interfaces/>

⁸<http://dblp.uni-trier.de/xml/>

⁹<https://wiki.dbpedia.org/Datasets>

¹⁰<http://freebase-easy.cs.uni-freiburg.de/dump/>

where $M(MD)$ and $M(DM)$ are the adjacency matrices mapping the relationships between movie and director vertices.

Additionally, in Table 5 we provide statistics of the used sparse matrices in each HIN including the the matrix dimensions (square matrix) and the average number of NNZ and non-zero rows.

Table 5: Matrix statistics by dataset.

Dataset	Dimension	NNZ	Non-zero rows
FourSquare	43.2K	101,369	18,417
IMDB	471.8K	173,657	97,620
DBLP	490.7K	357,272	138,597
DBPedia	9.0M	46,613	24,919
FreeBase	29.1M	321,244	192,767

8.3 Additional Experiments

8.3.1 Sparsity Estimation on SuiteSparse Collection [28]. We conduct experiments on various sparse inputs in the scientific computing area from the SuiteSparse matrix collection [28], which includes not only full graphs but also matrices used in optimization problems, as shown on Table 6. Using these matrices, we have constructed 2,000+ heterogeneous chains with different sparse matrices and varying sparsity.

Table 6: Sparse matrices used from SuiteSparse [28].

Matrix Name	# of row	# of col	NNZ	Area
GL7d17	1.5M	0.9M	25M	Optimization Problem
GL7d18	1.9M	1.5M	35M	Optimization Problem
GL7d19	1.9M	1.9M	7.3M	Optimization Problem
GL7d20	1.4M	1.9M	29M	Optimization Problem
GL7d21	0.8M	1.4M	18M	Optimization Problem
GL7d22	0.3M	0.8M	18M	Optimization Problem
GL7d23	0.1M	0.3M	2.6M	Optimization Problem
Rucci1	1.9M	1M	7.7M	Least Squares Problem
cont11_1	1.4M	1.9M	5.3M	Linear Programming
web-BerkStan	0.6M	0.6M	7.6M	Directed Graph
webbase-1M	1M	1M	3.1M	Weighted Directed Graph
wheel_601	0.9M	0.7M	2.1M	Combinatorial Problem
wikipedia-20051105	1.6M	1.6M	19M	Directed Graph
wikipedia-20070206	3.5M	3.5M	45M	Directed Graph

8.3.2 Comparison to Baselines in Robust Query Processing. : Given the connection between SMM and acyclic query processing, we conduct experiments by adding three new competitors in Robust Query Processing, introduced as follows:

- **Yannakakis algorithm (YK)** [53]: it achieves instance-optimal acyclic query processing by performing two exact semi-join passes (forward and backward) to eliminate all dangling tuples. We implemented it using the semi-join operator in DuckDB v1.2 [39].
- **Predicate Transfer (PT)** [52]: it generalizes Bloom join to multi-join queries by propagating lightweight Bloom filters along arbitrary join graphs in two passes. We used the bloom filter with a 2 bit hash bucket.
- **Parachute (PC)** [49]: it pre-computes and stores compressed fingerprint columns on foreign-key tables, enabling single-pass bidirectional information passing at query time with extra storage space. We used the official implementation and set $pbw = 16$.

As CoreMat accepts a chain of sparse matrices rather than relational tables as input, we constructed a shared pipeline to convert each sparse matrix A into a table T_A . Specifically, each entry $(row_{idx}, col_{idx}, value_{idx})$ is mapped to a key-value pair $(src = row_{idx}, dst = col_{idx})$ in a two-column table, and the join query $T_{A_1} \bowtie T_{A_2}$ is expressed as $T_{A_1}.dst = T_{A_2}.src$. By this conversion method, we can convert the HIN to a chain of tables and compare end-to-end speed-up of CoreMat to the above methods. For the queries in this experiment, we employ the range predicate “between” on tables with numeric columns. We generated percentiles at {5%, 30%, 50%} as the between range. Note that Parachute includes a pre-computation stage that processes a fixed set of tables and a fingerprint Parachute column, so we used tables derived from a fixed set of 20 sparse matrices and generated a total of 180 HINs (queries) of lengths [3, 4, 5, 6] from this set.

We compare the end-to-end execution runtime of the above methods and our CoreMat extraction algorithm CoreMat- Algo in Figure 20. Clearly, CoreMat is faster than all the competitors, whereas YC and PT achieve comparable performance and PC has the worst performance. This is because for PC, it is the only method in our experiments that has an off-line pre-compute stage, its pre-compute, rewrite, and fingerprint-build processes incur additional time that cannot be amortized in our experiment, making it slower than all the other methods. For YC and PT, they either use the semi-join operator to remove all the dangling tuples, or use bloom join operator to approximately remove the dangling tuples, and both of them use hash-based operators which is slower than 1D set intersection operator that is used in CoreMat- Algo, making them slower than our method.

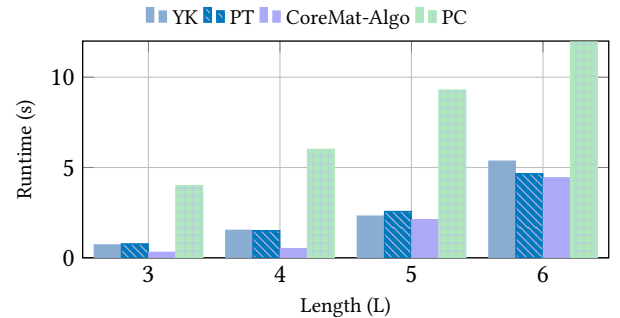


Figure 20: Runtime of different methods.

8.3.3 Case Study. We provide real-world use cases by examining the speed up of GNN training via CoreMat. We use the official implementation of directed graph neural networks, i.e., Dir-GCN and Dir-GAT from [43], as backbones, and insert CoreMat as a plug-in to sparsify the adjacency matrices in the training process. The SMM kernel used is torch-sparse [38] with cuSPARSE backend [36]. We compare the training time (ms) per epoch on four datasets used in [43]: Chameleon, Squirrel, Arxiv-Year and Snap-Patents. The hidden dimension is set to 64, training epoch is set to 100, and the number of layers varies from {2, 4, 8}. For all datasets except Snap-Patents, we train the GNNs on a single NVIDIA RTX 3090 GPU with 24 GB of memory. For Snap-Patents, we train the GNNs on a single NVIDIA H100 GPU with 80 GB of memory.

The results are presented in Table 7, where each dataset’s name is accompanied by its number of nodes. Clearly, by using CoreMat, we can effectively reduce the per-epoch training time for GNN training on most cases, especially on deeper models and larger graphs, e.g. Arxiv-Year and Snap-Patents. The speedup could reach up to 2.04×

Table 7: Training time per epoch (ms) w/o vs. w/ CoreMat.

Model	# of layers	Chameleon (2.2K)			Squirrel (5.2K)			ArXiv-Year (169.3K)			Snap-Patents (2.9M)		
		w/o	w/	Speedup	w/o	w/	Speedup	w/o	w/	Speedup	w/o	w/	Speedup
Dir-GCN	2	3.395	3.267	1.039	8.007	7.571	1.058	22.268	19.647	1.133	463.372	425.728	1.088
	4	5.341	4.167	1.282	9.080	8.997	1.009	41.355	32.774	1.262	653.979	530.930	1.232
	8	6.767	6.344	1.067	12.092	9.781	1.236	80.270	57.305	1.401	1034.640	683.943	1.513
Dir-GAT	2	7.978	7.324	1.089	9.120	8.663	1.053	37.726	33.005	1.143	560.786	487.147	1.151
	4	16.036	15.985	1.003	17.696	17.749	0.997	78.008	60.788	1.283	1259.913	852.735	1.477
	8	31.634	29.107	1.087	34.409	32.198	1.069	159.063	107.763	1.476	2658.220	1302.198	2.041

Note that for very few cases with Dir-GAT as the backbone model, using CoreMat cannot speed up the training, because the matrices’ dimensions are small, leading to a reduction in the proportion of matrix multiplication time in the total training time. Therefore, the construction time of CoreMat cannot be offset, making it be slower than its counterpart.

8.4 Connection to Acyclic Join Query

Both matrix multiplication and relational join query belong to the class of mathematical problems known as the *Generalized Sum-Product Problem over Semirings* [9, 44]. Below, we provide a detailed analysis of these connections along with the unique contributions of CoreMat, summarized in Table 8.

• **SMM and acyclic join processing can be unified under the semiring framework.** Both problems can be formulated within the Generalized Sum-Product Problem over **Semirings** $\mathcal{S} = (K, \oplus, \otimes, 0, 1)$ [44], where $\oplus$ and $\otimes$ satisfy associativity, commutativity, and distributivity, 0 is the identity element for $\oplus$, and 1 is the identity element for $\otimes$. Concretely, the two problems instantiate this framework with different semirings:

- Numerical SMM operates on the Arithmetic Semiring $(\mathbb{R}, +, \times)$, where both the structure and floating-point values are critical.
- Relational acyclic join operates on the Boolean Semiring $(\{0, 1\}, \vee, \wedge)$, focusing on the existence of tuples.

In graph-theoretic terms, a matrix chain corresponds to a path query, whose treewidth is $tw = 1$. Therefore, we agree that CoreMat is conceptually related to the Yannakakis algorithm: while Yannakakis provides a general solver for arbitrary acyclic joins, CoreMat can be regarded as a specialized realization for the $tw = 1$ path-query case. Nevertheless, the SMM setting has different computational characteristics from relational acyclic query processing. Prior studies [6, 49] show that Yannakakis algorithm may be up to $5\times$ slower due to the overhead of multiple passes over the input relations. For SMM, performance depends not only on structural pruning, but also on floating-point computation and I/O costs. Such factors are explicitly considered in SMM cost models [26] and are empirically reflected in Figure 11 of our paper, which shows the end-to-end SMM speedup achieved by CoreMat.

• **CoreMat as an SMM-specialized realization of Yannakakis-style reduction.** Since the “removable elements” identified by CoreMat are mathematically analogous to the “dangling tuples” eliminated by the Yannakakis algorithm, we can use the Yannakakis algorithm’s to compute CoreMat. Nevertheless, as a specialized instantiation of Yannakakis-style reduction designed for SMM, CoreMat introduces three distinctive contributions:

- (1) **Theoretical grounding.** CoreMat provides a graph-theoretic characterization of the fully reduced state via D-core [33]: for a matrix chain, we theoretically prove that its CoreMat shares the same set of edges with the $(1, 1)$ -core of its multipartite graph. This characterization explains the elimination of all dangling elements via the maximal of D-core, connects SMM to D-core decomposition.
- (2) **Algorithmic design.** Given the above theoretical grounding, established serial, parallel, and distributed D-core algorithms, including peeling-based [25] and H-index based methods [30], can be adapted for computing CoreMat computation. In other words, we do not need to solely rely on “two-pass reduction” idea of Yannakakis algorithm.
- (3) **Lightweight implementation.** Yannakakis-style reduction is typically implemented via relational semi-joins, using sorting or hashing over tuples. In contrast, CoreMat operates directly on COO-format sparse matrices and replaces such relational operators with lightweight 1D set intersections over index arrays.

• **Connection to Robust Query Processing (RQP).** Indeed, our work is closely aligned with the philosophy of RQP. A key challenge in RQP is to avoid “cardinality explosion”, i.e., unpredictable growth of intermediate results caused by inaccurate estimates. Existing SMM methods also face a similar issue caused by “structural noise”: non-zero elements that survive intermediate computations but ultimately do not contribute to the final result. By using CoreMat as a deterministic structural guardrail, we reduce the computation to the necessary core before the expensive numerical phase begins, thereby reducing memory usage and FLOPs. This makes SMM computation more robust to various input sparsity matrices and helps prevent worst-case runtime blowups.

Table 8: Yannakakis algorithm v.s. CoreMat.

Feature	Yannakakis algorithm	CoreMat (Our work)
Domain	Relational database theory	SMM and graph analytics
Algebraic basis	Boolean Semiring ($\{0, 1\}, \vee, \wedge$)	Arithmetic Semiring ($\mathbb{R}, +, \times$)
Target operation	Acyclic join Queries	SMM
Topological scope	General acyclic trees	Linear chains
Mechanism	Relational semi-joins ($R \ltimes S$)	Parallel 1D set intersections
Redundancy target	Dangling tuples	Removable elements
Guiding principle	Full semi-join reduction	$(1, 1)$ -core of D-multi-partite Graphs
Robustness focus	Bound intermediate relation size	Robust execution: Bound FLOPs and memory

8.5 Proofs

In this section, we restate the Lemmas/Theorems/Claims and provide the proofs.

8.5.1 Proof of Lemma 9.

LEMMA 9. For a random variable sequence $\{x_i\}_{i=1}^k$ independently sampled from uniform distribution $U(0, 1)$, denote $x_{(i)}$ as its order statistic, i.e., $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(k)}$. Then the sample range $r = x_{(k)} - x_{(1)}$ follows a Beta distribution [42] with shape parameters $\alpha = k - 1$, $\beta = 2$, denoted by $Be(k - 1, 2)$.

PROOF. We can extract the joint probabilistic distribution of $x_{(1)}$ and $x_{(k)}$ by:

$$f(x_{(1)}, x_{(k)}) = k(k-1)p(x_{(1)})p(x_{(k)})(F(x_{(k)}) - F(x_{(1)}))^{k-2} \quad (32)$$

Let $r = x_{(k)} - x_{(1)}$, $z = x_{(1)}$, the inverse transform is $x_{(k)} = r + z$, $x_{(1)} = z$, then $\det\left(\frac{\partial}{\partial(r,z)} \begin{bmatrix} r+z \\ z \end{bmatrix}\right) = 1$, hence we have:

$$f(r, z) = k(k-1)p(z)p(z+r)(F(z+r) - F(z))^{k-2} \quad (33)$$

Then integrate by z :

$$f(r) = \int_{-\infty}^{+\infty} k(k-1)p(z)p(z+r)(F(z+r) - F(z))^{k-2} dz \quad (34)$$

Notice that in the case of uniform distribution, $0 \leq x_{(1)} \leq x_{(k)} \leq 1$ leads to $r - 1 \leq 0 \leq z \leq r \leq 1$. Then

$$f(r) = k(k-1)x^{k-2}(1-x) \sim Be(k-1, 2) \quad (35)$$

□

8.5.2 Proof of Theorem 7.

THEOREM 10. Given a random variable sequence $\{x_i\}_{i=1}^k$, where each x_i is sampled independently from $Uniform(0, 1)$ and k is unknown. Assume we have access to $x_{\max} = \max\{x_i\}_{i=1}^k$ and $x_{\min} =$

$\min\{x_i\}_{i=1}^k$, then we can estimate k with the sample range r by

$$\hat{k} = \frac{2}{1-r} - 1 \quad (36)$$

where $r = x_{\max} - x_{\min}$

PROOF. By Lemma 9, we have $r \sim Be(k-1, 2)$. The first moment and variance of Beta distribution have the below closed form solutions:

$$E(r) = \frac{k-1}{k+1} = 1 - \frac{2}{k+1} \quad (37)$$

$$Var(r) = \frac{2(k-1)}{(k+1)^2(k+2)} \quad (38)$$

By solving Eq (37) for k , we have $\hat{k} = \frac{2}{1-r} - 1$. □

8.5.3 Proof of Claim in Theorem 8.

LEMMA 11 (PROOF OF CLAIM EQ (18)). Let $\epsilon' = 2 - \frac{4}{\epsilon+2}$, we have

$$\left|\bar{r} - \frac{k-1}{k+1}\right| \geq \frac{1}{k+1}\epsilon' \quad (39)$$

PROOF. We prove by raising a contradiction, assume that:

$$\left|\bar{r} - \frac{k-1}{k+1}\right| < \frac{1}{k+1}\epsilon'$$

Then,

$$\frac{2}{k+1} - \frac{1}{k+1}\epsilon' \leq 1 - \bar{r} \leq \frac{1}{k+1}\epsilon' + \frac{2}{k+1} \quad (40)$$

Notice that,

$$k(1 - \bar{r}) \geq \frac{1}{2}(k+1)(1 - \bar{r}) \geq \frac{1}{2}(2 - \epsilon') \quad (41)$$

By combining equation (18) and (19), we yield the following contradiction:

$$\left|\frac{\hat{k} - k}{k}\right| \leq \frac{\frac{1}{k+1}\epsilon'(k+1)}{\frac{1}{2}(2 - \epsilon')} = \frac{2\epsilon'}{2 - \epsilon'} = \epsilon \quad (42)$$

Hence we proved the claim. □